A Project report on

AI-DRIVEN PERSONALIZED FIT RECOMMENDATION SYSTEM USING COMPUTER VISION

In partial fulfilment of the requirements for Award of Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

U. PADMA JAYASRI	**22B91A05S9**
S. VASUDEVARAO	**22B91A05R5**
S. VASANTH	**22B91A05P8**
S. AMBICA MALIKHARJUNA RAO	**22B91A05P1**
S. AHMAD VALI AFRID	**22B91A05P9**

Under the Esteemed Guidance of

Dr. GADIRAJU MAHESH

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

S.R.K.R. ENGINEERING COLLEGE (A)

Chinna Amiram, Bhimavaram, West Godavari Dist., A.P.

[2025 – 2026]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

S.R.K.R. ENGINEERING COLLEGE (A)

Chinna Amiram, Bhimavaram, West Godavari Dist., A.P.

[2025 – 2026]

BONAFIDE CERTIFICATE

This is to certify that the project work entitled **AI-DRIVEN PERSONALIZED FIT RECOMMENDATION SYSTEM USING COMPUTER VISION** is the bonafide work of **U. PADMA JAYASRI (22B91A05S9), S. VASUDEVARAO (22B91A05R5), S. VASANTH (22B91A05P8), S. AMBICA MALIKHARJUNA RAO (22B91A05P1), S. AHMAD VALI AFRID** who carried out the project work under my supervision in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING.

SUPERVISOR
Dr. GADIRAJU MAHESH
Professor

HEAD OF THE DEPARTMENT
Dr. Bh. V. S. RAMA KRISHAM RAJU
Professor

SELF DECLARATION

We hereby declare that the project work entitled **AI-DRIVEN PERSONALIZED FIT RECOMMENDATION SYSTEM USING COMPUTER VISION** is a genuine work carried out by us in B.Tech., Computer Science and Engineering at SRKR Engineering College(A), Bhimavaram and has not been submitted either in part or full for the award of any other degree or diploma in any other institute or University.

U. PADMA JAYASRI	**22B91A05S9**
S. VASUDEVA RAO	**22B91A05R5**
S. VASANTH	**22B91A05P8**
S. AMBICA MALIKHARJUNA RAO	**22B91A05P1**
S. AHMAD VALI AFRID	**22B91A05P9**

ACKNOWLEDGEMENTS

I sincerely express my gratitude to **SRKR Engineering College** for providing me with the opportunity to undertake this final year project, "**AI-DRIVEN PERSONALIZED FIT RECOMMENDATION SYSTEM USING COMPUTER VISION**".

I extend my heartfelt thanks to our **Principal, Dr. K. V. Murali Krishna Raju**, for fostering an academic environment that encourages research and innovation. I am also deeply grateful to **Dr. Bh. V. S. Rama Krishnam Raju**, Head of the Department of Computer Science & Engineering, for continuous support and motivation throughout this project.

A special thanks to my project supervisor, **Dr. Gadiraju Mahesh, Professor** whose **guidance, technical expertise, and invaluable feedback** have been instrumental in shaping this work. His mentorship has provided me with both knowledge and confidence to tackle challenges effectively.

I also appreciate the support of **faculty members, technical staff, project teammates, family, and friends**, whose encouragement and insights have been invaluable.

U. PADMA JAYASRI	**22B91A05S9**
S. VASUDEVA RAO	**22B91A05R5**
S. VASANTH	**22B91A05P8**
S. AMBICA MALIKHARJUNA RAO	**22B91A05P1**
S. AHMAD VALI AFRID	**22B91A05P9**

ABSTRACT

Inconsistent sizing across fashion brands remains a major challenge in online retail, leading to poor fit, frequent returns, and customer dissatisfaction. To address this issue, an AI-driven Personalized Fit Recommendation System is proposed which uses computer vision and image processing to accurately capture a user's body dimensions from uploaded or live images. Using OpenCV and edge detection, the system measures the body from images, and machine learning models such as Linear Regression, Random Forest, and Decision Tree are applied to analyze the extracted measurements and improve prediction accuracy by learning patterns from historical data. The processed measurements are then compared with brand-specific sizing charts to recommend the best-fitting clothing size. This unified solution reduces return rates and enhances the overall shopping experience by delivering precise, personalized size recommendations across multiple product categories.

S. No.	CONTENTS		Page. No.
	ABSTRACT		i
	CONTENTS		ii
	LIST OF TABLES		iv
	LIST OF FIGURES		v
1	INTRODUCTION		1
2	LITERATURE REVIEW		4
3	PROBLEM STATEMENT		11
3	METHODOLOGY/SYSTEM DESIGN		
	3.1	EXISTING SYSTEM	12
	3.2	PROPOSED SYSTEM	13

	3.3	REQUIREMENTS	14
	3.4	SYSTEM ARCHITECTURE	16
	3.5	DESIGN DIAGRAMS/PROPOSED MODEL	24
4	IMPLEMENTATION AND TESTING		
	4.1	IMPLEMNETATION APPROACH	31
	4.2	FUNCTIONAL ANDALGORITHMIC IMPLEMENTATIONS	31
	4.3	INTEGRATION DETAILS	33
	4.4	TESTING 4.4.1 TEST CASES	35
5	DISCUSSION OF RESULTS		36

6	CONCLUSION & FUTURE SCOPE	38

7	REFERENCES	39
8	APPENDIX 1: CODE	41
	APPENDIX 2: SCREENS	136

S. No.	LIST OF TABLES	Page No.

Table 2.1	List of Literatures	9
Table 4.4.1.1	Test cases	36
Table 5.1	Accuracy metrics	37

S. No.	LIST OF FIGURES	Page No.
Figure 1.1	Introduction Process Overview	3
Figure 3.2.1	Proposed System	14
Figure 3.4.1	System Architecture	16
Figure 3.5	Proposed model	24
Figure 3.5.1.3	Business Flow Diagram	32
Figure 3.5.1.4	Canny Edge detection	32
Figure 5.4	Comparision of model performance	40
Figure 4.6.1	User DashBoard	150
Figure 4.6.2	Manual input of height and other details	150
Figure 4.6.3	Select the image	151
Figure 4.6.4	Processing Steps of Image	151

Figure 4.6.5	**Body Measurements**	**152**
Figure 4.6.6	**Recommendation System**	**152**
Figure 4.6.8	**History Timeline**	**140**

1. INTRODUCTION

The rapid growth of **e-commerce platforms** has transformed the global retail industry, especially in the **fashion and apparel sector**. With widespread internet and smartphone usage, **online shopping** has become highly popular due to its **convenience, speed, and wide product availability**. However, a major unresolved issue remains: **incorrect clothing size selection**, since customers cannot try garments before purchase.

The main problem in online apparel systems is **size mismatch**. Most platforms rely on **static size charts** based on general body measurements, which fail to consider individual differences like **body shape, posture, and proportions**. Moreover, sizing varies across brands, where the same size label may represent different measurements, leading to confusion and incorrect choices.

This results in **poorly fitting clothes**, causing **customer dissatisfaction, returns, and exchanges**. For businesses, this increases **return rates, logistics costs, and operational burden**, while reducing **customer trust and loyalty**. Hence, solving size prediction issues is important for both **user experience and business efficiency**.

Another limitation is the lack of **accurate body measurement methods**. Manual measurement using tapes is **inconvenient and error-prone**, while advanced **3D body scanning systems** are accurate but expensive and impractical for large-scale use. Therefore, there is a need for a **low cost, accessible, and automated solution** using simple inputs like images.

Recent advances in **Computer Vision and Machine Learning** provide effective solutions. **Computer Vision** helps extract visual features from images, while **Machine Learning** enables prediction from learned patterns. Together, they allow estimation of **human body measurements from images** without special hardware.

This problem is highly relevant to the **fashion e-commerce industry**, where improved size prediction can significantly enhance **customer satisfaction**, reduce **return rates**, and improve **supply chain efficiency**. It also increases **user confidence** in online shopping platforms.

The motivation for this project comes from the common real-world issue of **incorrect clothing sizes** faced by online shoppers. Technically, it provides an opportunity to integrate **image**

processing, feature extraction, and machine learning using tools like **OpenCV, MediaPipe, and Scikit-learn**.

To solve these issues, the proposed system is an **AI-Driven Personalized Fit Recommendation System**. It takes a **full-body image** as input through a web interface and applies preprocessing techniques such as **noise reduction (Gaussian Blur)**, **contrast enhancement (CLAHE)**, and **background removal**.

Next, **feature extraction** is performed using **Canny Edge Detection, Contour Detection, Convex Hull, and Bounding Box methods**. In parallel, **MediaPipe Pose estimation** detects key body landmarks, improving accuracy in identifying body structure.

From these features, the system calculates **body measurements** such as **chest, waist, hips, and shoulder width** using **Euclidean distance** and **pixel-to-real-world scaling**, often supported by user height input. A hybrid approach combining **landmark and contour analysis** improves reliability.

These measurements are then fed into **Machine Learning models** including **Linear Regression, Decision Tree, and Random Forest**. Each model contributes differently: regression captures relationships, decision trees apply rules, and random forest improves accuracy through ensemble learning.

The final output is mapped to **brand-specific size charts**, generating a recommended size along with **fit type (tight/regular/loose)** and a **confidence score**. These results are displayed through a simple user interface.

The scope of this project includes a complete pipeline of **image preprocessing, feature extraction, measurement estimation, and ML-based prediction**. It is a scalable prototype but currently limited to **2D image processing**, without **3D scanning** support. Future improvements may include **real-time processing and deep learning integration**.

The objectives are to develop an accurate **size prediction system**, apply **Computer Vision techniques**, and use **Machine Learning models** to improve recommendation accuracy. The system also aims to reduce **return rates and improve user experience**.

In conclusion, this project demonstrates how **Artificial Intelligence, Computer Vision, and Machine Learning** can solve the real-world problem of **online clothing size mismatch**, improving both **customer satisfaction and business efficiency** in the e-commerce domain.

Fig 1.1 : Introduction Process Overview

2. LITERATURE REVIEW

Recent advancements in **Artificial Intelligence (AI), Computer Vision, and Machine Learning (ML)** have significantly transformed the domain of **online fashion and apparel recommendation systems**, particularly in addressing the long-standing problem of **incorrect clothing size selection**. In modern e-commerce platforms, customers frequently face difficulty in choosing the correct size due to the **absence of physical trial rooms, inconsistent sizing standards across brands, and lack of personalized recommendations**. These issues often result in **high product return rates, increased logistics costs, and reduced customer satisfaction**, making size recommendation a critical research area.

Rahman et al. **[1]** (2021) introduced a system that integrates **Internet of Things (IoT) with computer vision** to automatically extract human body measurements from images. Their approach uses image preprocessing, edge detection, and feature extraction techniques to determine body dimensions and map them to size charts. This work demonstrates the effectiveness of combining **vision-based measurement systems with smart retail environments**, significantly reducing human dependency. However, the system performs well mainly in **controlled environments**, and its accuracy is affected by **lighting conditions, camera angles, and user posture**, which limits real-world applicability.

Kim et al. **[2]** (2022) proposed a more advanced system that utilizes both **2D image data and 3D point cloud information** to estimate garment sizes. By leveraging depth information, the system reconstructs garment geometry and extracts precise measurements. This hybrid approach significantly improves measurement accuracy compared to traditional 2D methods. Despite its advantages, the requirement of **specialized hardware such as LiDAR sensors or depth cameras**, along with high computational complexity, makes the system **expensive and difficult to deploy at scale**, especially for common users.

Thota et al. **[3]** (2022) focused on overcoming hardware limitations by proposing a deep learning model capable of estimating **3D human body shape from simple 2D images** captured from frontal and side views. This approach eliminates the need for physical scanners while still providing accurate measurement estimation. The system uses neural networks to reconstruct a parametric body model and extract anthropometric features. Although promising, this method requires

multiple input images and high computational resources, making it less suitable for **real-time applications and mobile-based systems**.

Alhassawi et al. **[4]** (2023) explored the application of **machine learning algorithms** for clothing size prediction using anthropometric data obtained from **3D body scans**. Their study compares multiple algorithms such as **Random Forest, Support Vector Machines (SVM), and K-Nearest Neighbours (KNN)** to determine the most effective model for size prediction. The results demonstrate that ML-based approaches can significantly improve prediction accuracy. However, the reliance on **large-scale 3D datasets and expensive data collection methods** makes this approach impractical for general users.

Jogi et al. **[5]** (2023) proposed an AI-based system that focuses on **body type classification** instead of exact measurement extraction. Using **Convolutional Neural Networks (CNNs)**, the system categorizes users into predefined body types such as slim, average, or plus-size, and provides fashion recommendations accordingly. While this improves personalization in terms of style, it does not address the core problem of **precise size prediction**, limiting its effectiveness in reducing product returns.

Recent advancements in deep learning are reflected in the work of Chen et al. **[6]** (2025), who introduced a **CNN-ResNet-based model** for accurate anthropometric measurement extraction. Their system focuses on identifying key body regions and learning spatial relationships between body parts to predict measurements with high precision. Although the model achieves excellent accuracy, it requires **high-performance GPUs and large training datasets**, making it unsuitable for lightweight deployment.

Similarly, Zhao et al. **[7]** (2025) proposed MeasureXpert, a system that extracts measurements from **partial 3D body scans** by reconstructing a complete body model. This approach addresses the limitation of requiring full-body scans and allows flexibility in pose variations. However, it still depends on **3D scanning data and complex reconstruction pipelines**, which are not feasible for everyday users.

Li et al. **[8]** (2025) addressed one of the most challenging problems in this domain: **body estimation under clothing**. Their ETCH model uses advanced deep learning techniques to estimate the underlying body shape even when users wear loose or layered garments. This improves real-world applicability but comes at the cost of **high computational complexity and lack of real-time performance**.

From the overall analysis of existing literature, it is evident that most systems focus either on **high accuracy using complex 3D models** or **lightweight approaches with limited precision**. There is a clear research gap in developing a system that is **accurate, efficient, cost-effective, and scalable for real-world applications**. This gap motivates the development of the proposed **AI Driven Personalized Fit Recommendation System**, which leverages **2D image processing, efficient computer vision techniques, and ensemble machine learning models** to achieve a balance between performance and practicality.

S. No	Paper Title	Publisher & Year	Key Findings	Limitations Identified
1	Toward Smart IoT for Apparel Retail Industry: Automatic Customer's Body Measurements and Size Recommendation System using Computer Vision	IEEE Access, 2021	• Uses computer vision techniques for automatic body measurement extraction from images • Integrates IoT-enabled smart retail system for real-time processing • Reduces manual measurement errors and improves personalization • Maps extracted measurements with brand-specific size charts	Highly dependent on lighting conditions, camera angle, and user posture • Works mainly in controlled environments • Limited handling of loose clothing and complex backgrounds
2	Automatic Measurements of Garment Sizes Using Computer Vision Deep	MDPI Applied Sciences, 2022	Combines 2D images and 3D point cloud data for accurate garment measurement • Uses deep learning for	Requires depth cameras or LiDAR sensors • High computational cost

	Learning Models and Point Cloud Data		feature extraction and geometric modeling • Provides higher accuracy compared to traditional 2D methods • Improves reliability of automated size estimation systems	due to 3D processing • Not suitable for low-end devices or mobile applications
3	Estimating 3D Body Shape and Clothing Measurements from Frontal and Side-View Images	arXiv, 2022	Reconstructs full 3D body shape using only 2D images • Eliminates need for expensive 3D scanning hardware • Enables extraction of realistic body and clothing measurements • Supports scalable virtual fitting systems	Requires multiple images (front and side view) • High processing time and computational complexity • Accuracy affected by pose and lighting conditions
4	Evaluating Machine Learning Models for Clothing Size Prediction Using Anthropometric Measurements from 3D Scanning	Nature Scientific Reports, 2023	Uses machine learning models like Random Forest, SVM, and KNN for size prediction • Compares multiple algorithms to identify best-performing model • Improves prediction accuracy using real anthropometric data	Requires large datasets of 3D body scans • High data collection cost and infrastructure requirement • Not suitable for real-time imagebased systems

			• Reduces size mismatch and product returns	
5	AI-Powered Body Type Analysis for Fashion Recommendation System	IJIST, 2023	Uses CNN for body type classification from images • Categorizes users into predefined body types (slim, average, plus-size) • Provides personalized fashion recommendations based on body type • Enhances user experience in online shopping	Does not compute exact body measurements • Limited to predefined categories • Not suitable for accurate size prediction
6	A Focused Human Body Model for Accurate Anthropometric Measurements Extraction	IEEE CVPR, 2025	Uses CNN-ResNet architecture for precise body measurement extraction • Learns spatial relationships between body parts • Provides high accuracy in anthropometric prediction • Improves robustness against pose variations	Requires high GPU resources for training and inference • High computational complexity • Not optimized for real-time mobile applications

7	MeasureXpert: Automatic Anthropometric Measurement Extraction	IEEE ICCV, 2025	Extracts measurements from partial 3D scans • Uses deep learningbased scan fusion for	Requires 3D scan data • Complex reconstruction
	from Two Unregistered Partial Scans		reconstruction • Supports arbitrary poses without requiring fullbody scans • Improves flexibility in measurement estimation	pipeline • High memory and computational requirements
8	ETCH: Generalizing Body Fitting to Clothed Humans via Equivariant Tightness	arXiv, 2025	Estimates underlying body shape under clothing • Uses equivariant learning for better generalization • Handles loose and layered garments effectively • Improves realism in body fitting systems	Computationally expensive 3D fitting process • Not suitable for real-time systems • Requires highquality training datasets

Table 2.1 : List of Literatures

SUMMARY

[1] by Rahman et al. presents an IoT and computer vision-based system for automatic body measurement extraction and clothing size recommendation. The study demonstrates how image processing techniques can reduce dependency on manual measurement and improve online shopping accuracy. However, the system is sensitive to environmental conditions such as lighting, posture, and camera angle, limiting real-world robustness.

[2] by Kim et al. proposes a hybrid 2D–3D deep learning approach using point cloud data for garment size estimation. The model achieves higher accuracy by reconstructing human body geometry using depth information. However, it requires expensive hardware and is not suitable for low-cost or real-time applications.

[3] focuses on estimating human body measurements using 3D reconstruction from multiple image views. The approach provides accurate anthropometric estimation without physical scanning devices. However, it requires multiple images and has high computational complexity, making it less practical for real-time systems.

[4] by Alhassawi et al. evaluates multiple machine learning models for clothing size prediction using anthropometric data from 3D scans. The study shows that ML models like Random Forest improve prediction accuracy. However, the reliance on 3D scan datasets limits real-world usability.

[5] by Jogi et al. introduces a CNN-based body type classification system for fashion recommendation. It improves personalization by categorizing users into body types and suggesting suitable clothing styles. However, it does not provide exact size prediction, limiting its precision.

[6] by Chen et al. proposes a deep learning CNN–ResNet model for extracting accurate body measurements from images. The system improves precision by focusing on key body regions. However, it is computationally expensive and not optimized for lightweight deployment.

[7] by Zhao et al. introduces a framework for reconstructing full human body shapes using partial 3D scans. The method improves measurement accuracy in incomplete scan conditions. However, it depends heavily on 3D scan data and is not suitable for image-only systems.

[8] by Li et al. presents a 3D body fitting approach that estimates body shape under clothing variations. The system improves robustness for real-world fashion scenarios. However, it is computationally heavy and not designed for real-time size recommendation systems.

PROBLEM STATEMENT

Inconsistent sizing across fashion brands remains a major challenge in online retail, leading to poor fit, frequent returns, and customer dissatisfaction. Traditional methods of size recommendations rely on static size charts that do not account for individual body shape, posture, or proportions, resulting in incorrect purchasing decisions. Furthermore, current systems lack personalization and often fail to adapt to brand-specific sizing standards, exacerbating the issue of sizing inconsistency. There is a need for an intelligent, accurate, and real-time system that can objectively analyze a customer's body dimensions using computer vision techniques and provide precise, personalized size recommendations for various products, including clothing, shoes, and accessories.

3.METHODOLOGY / SYSTEM DESIGN

3.1 Existing System

Existing systems for clothing size recommendation in online shopping platforms primarily depend on **static size charts (S, M, L, XL)** and simple rule-based methods that assume standard body proportions. Users manually select sizes based on judgment, past purchases, or basic filters like **gender, age group, and brand preferences**. Some systems also use basic **collaborative filtering / purchase history–based recommendation**, but they still do not analyze actual **body measurements** or **body shape variations**, making them largely non-personalized.

Although advanced approaches such as **3D body scanning**, **deep learning models**, and **computer vision techniques** exist, they are not widely used due to high cost, need for specialized hardware like **depth sensors / LiDAR**, and poor scalability in real-world e-commerce systems. Most existing solutions do not integrate **pose estimation**, **image-based body measurement extraction**, or **AI-based size prediction** in a unified framework. This results in issues like high **return rates**, inaccurate **fit prediction**, poor user experience, and customer dissatisfaction. Hence, a fully scalable, real-time **AI-driven personalized fit recommendation system** is still not effectively implemented in current real-world platforms.

Existing / Identified Gaps:

- **Lack of personalization**: Existing systems rely heavily on static size charts and do not offer personalized fit recommendations based on individual body measurements.
- **Inconsistent and inaccurate fit predictions**: Many existing systems fail to account for body proportions or brand-specific sizing variations, leading to inaccurate recommendations.
- **Limited real-time body measurement extraction**: Current approaches to measuring body dimensions are either manual or require specialized hardware, limiting their scalability for everyday online shopping.
- **Dependency on manual input**: Many platforms require users to input their body measurements manually, which is time-consuming and prone to errors.

3.2 Proposed System

The proposed system is an AI-driven Personalized Fit Recommendation System designed to solve the problem of inconsistent clothing sizes across different fashion brands and improve online apparel fit accuracy. The system uses **Computer Vision, Image Processing, and Machine Learning** to estimate body measurements from user-uploaded images and provide personalized size recommendations for clothing items such as shirts and pants. It replaces traditional static size charts with an intelligent prediction-based approach to reduce size mismatch and product returns.

In this system, the user uploads a full-body image along with an essential input parameter, **user height**, which acts as a scaling reference to convert pixel measurements into real-world dimensions. The image is processed using **OpenCV** for preprocessing tasks such as noise removal, contrast enhancement, and background segmentation. Feature extraction is performed using **edge detection (Canny), contour detection, and convex hull techniques**, along with **MediaPipe Pose Estimation** to identify key body landmarks. These extracted features are used to compute body measurements like shoulder width, chest, waist, hip, and leg proportions.

After measurement extraction, the system applies **Machine Learning models implemented using Scikit-learn**, including **Decision Tree, Random Forest, and Linear Regression**, to predict the most suitable clothing size. The models learn patterns between body measurements and standard size categories. The final output is generated by mapping predictions to **brand-specific size charts**, providing accurate size recommendations separately for shirts and pants, along with optional fit classification (tight, regular, or loose).

Overall, the system integrates **OpenCV, MediaPipe, NumPy, Pandas, and Scikit-learn** to build a complete end-to-end pipeline from image input to size recommendation. This approach improves personalization, reduces dependency on manual sizing, and enhances the online shopping experience by providing more accurate and reliable clothing fit predictions.

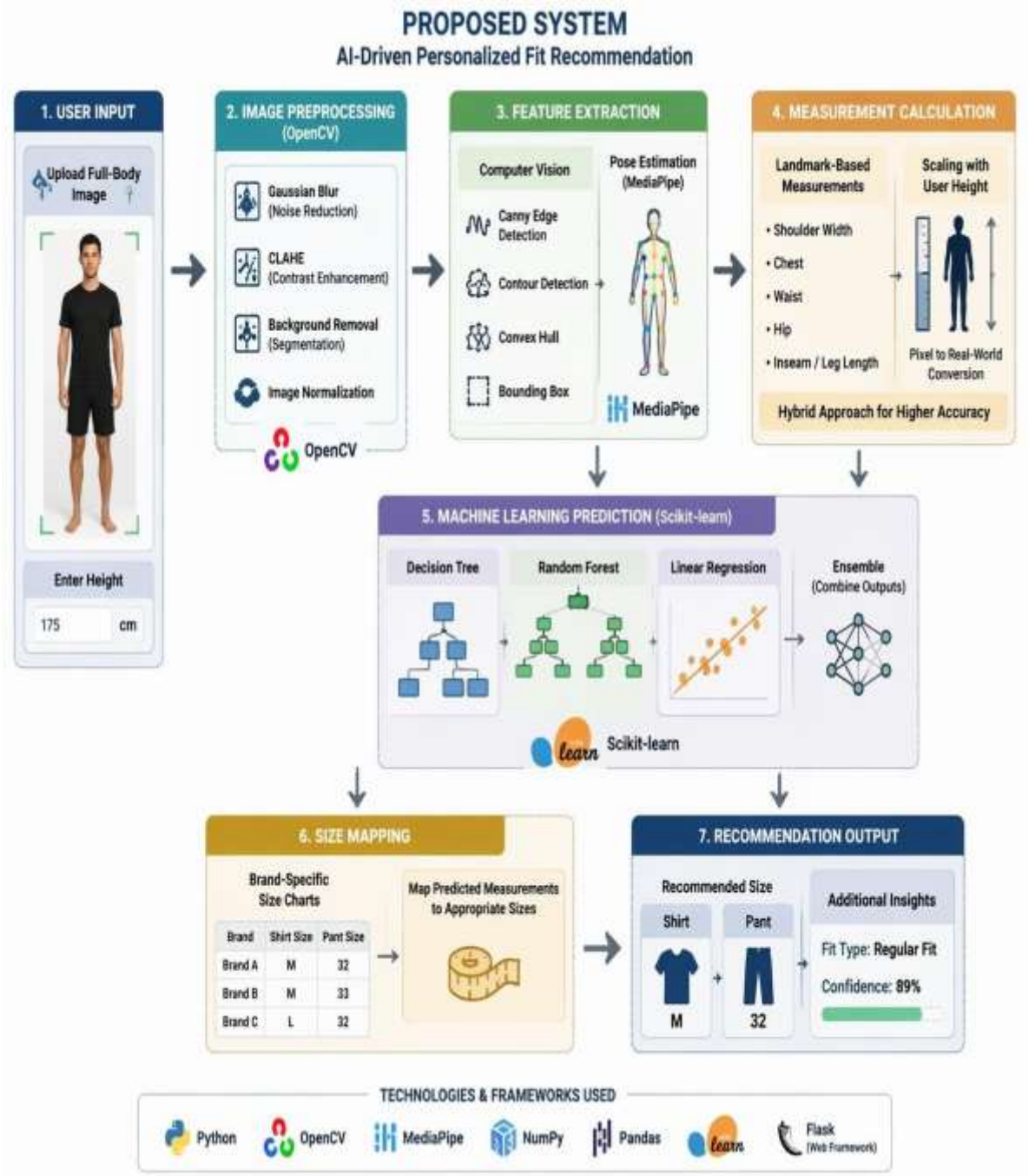


Figure 3.2.1: Proposed System

3.3 Requirements

3.3.1 Software Requirements

Operating System: Windows 10/11 / Ubuntu (Recommended for ML environment)

- **Programming Language:** Python 3.x
- **Frontend:** React.js, HTML, CSS, JavaScript
- **Backend Framework:** Flask (Python Framework)
- **Database:** MongoDB
- **Computer Vision Library:** OpenCV
- **Pose Estimation:** MediaPipe
- **Machine Learning Library:** Scikit-learn
- **Data Processing Libraries:** NumPy, Pandas
- **API Testing Tool:** Postman / CURL
- **Version Control:** Git, GitHub
- **Development Tool:** Visual Studio Code
- **Package Manager:** pip (Python Package Installer)

3.3.2 Hardware Requirements

Processor: Intel Core i5 / Ryzen 5 or higher

- **Memory (RAM):** Minimum 8 GB (16 GB recommended for better performance)
- **Storage:** Minimum 256 GB free disk space
- **System Type:** 64-bit operating system
- **Camera:** Required for capturing/uploading user images
- **Internet:** Required for installation of dependencies and model libraries
- **GPU (Optional):** NVIDIA GPU recommended for faster image processing and model training

3.4 System Architecture

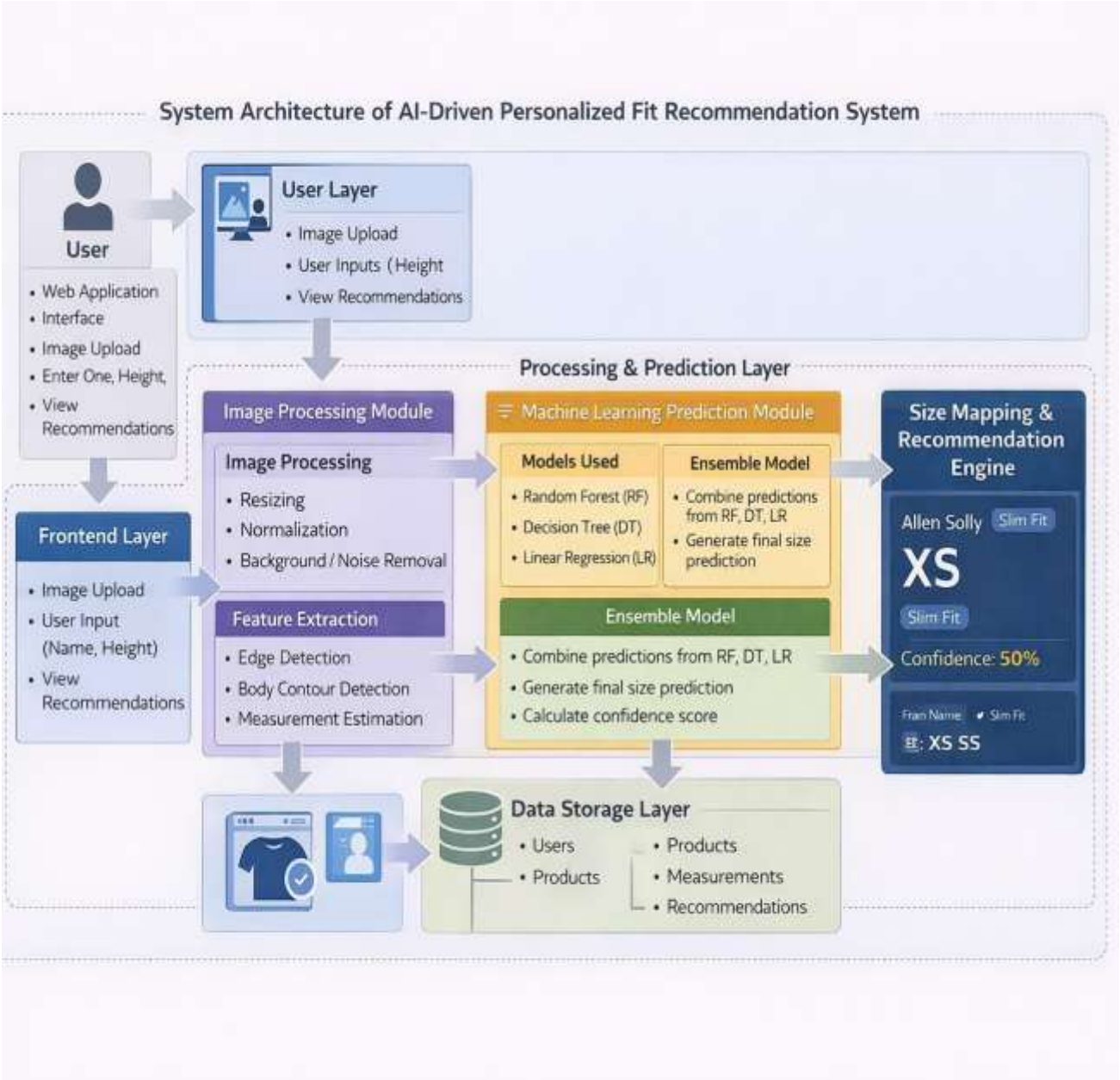


Figure 3.4.1: System Architecture

The proposed system is designed as an **intelligent, modular, and scalable AI-based architecture** that provides personalized footwear size recommendations by combining **user-provided images** with **explicit physical inputs such as height**. The system integrates **computer vision techniques**,

machine learning algorithms, and **rule-based decision mechanisms** to accurately extract measurements and generate reliable size predictions.

The architecture follows a **layered and service-oriented design**, where each module performs a specific function in the processing pipeline. This ensures **clear separation of concerns**, easier maintenance, scalability, and efficient data flow from user input to final recommendation output. The system is structured into multiple interconnected modules, each responsible for a critical stage of processing.

A. User Interface Layer (Frontend – React.js)

The User Interface Layer acts as the **primary interaction point** between the user and the system. It is implemented using **React.js**, ensuring a responsive and dynamic web experience.

This layer allows users to:

- Upload an image or capture it via webcam
- Enter **mandatory input such as height** (very important for scaling accuracy)
- Provide additional metadata like gender and preferences The UI also displays:
- Extracted measurements
- Recommended shoe sizes (UK/US/EU)
- Fit classification and product suggestions

The frontend is built using a **component-based architecture**, enabling modular UI development. It uses **state management (Context API)** and communicates with the backend via **RESTful APIs**. Real-time feedback such as loading indicators and error messages ensures a smooth and user friendly experience.

B. Image Acquisition Module

This module is responsible for collecting input data required for processing. The system supports:

- Image upload from local storage
- Real-time capture using a webcam

In addition to the image, the system **requires the user's height as input**, which plays a crucial role in improving measurement accuracy and proportional scaling.

Before processing, the module performs:

- Image format validation
- Quality checks (clarity, visibility of foot)

Once validated, the image and user inputs are forwarded to the backend processing pipeline.

C. Image Preprocessing Unit

The preprocessing module enhances the input image to improve the performance of computer vision algorithms. Raw images often contain noise, lighting variations, and distortions, which can affect measurement accuracy.

The following techniques are applied:

- **Color Space Conversion (RGB → HSV)** for better segmentation
- **Gaussian Blurring** to remove noise
- **Normalization** to standardize pixel values The objective of this module is to:
- Improve segmentation quality
- Enhance edge detection
- Prepare the image for accurate feature extraction

This stage is critical because the accuracy of all downstream modules depends on the quality of preprocessing.

D. Feature Extraction and Measurement Module (Core Module)

This is the **core computational component** of the system, where actual measurement extraction takes place using **computer vision techniques (OpenCV)**.

The processing pipeline includes:

I. **K-Means Clustering** → separates foreground (foot) from background
II. **Canny Edge Detection** → detects object boundaries
III. **Contour Detection** → identifies the exact shape of the foot
IV. **Bounding Box Extraction** → isolates the region of interest

Measurement Strategy

To convert pixel measurements into real-world units, the system uses:

V. A **reference object (A4 sheet)** for scale calibration
VI. **User height input** to improve proportional estimation

Using scaling factors, pixel dimensions are converted into **centimeters**, resulting in accurate estimation of:

VII. Foot length

This hybrid approach (reference object + height input) significantly improves measurement reliability compared to traditional methods.

E. Size Mapping and Recommendation Engine

This module acts as the **decision-making unit** of the system. It converts extracted measurements into meaningful recommendations.

Core functionalities include:

I. Mapping foot measurements to **standard size charts (UK, US, EU)**

II. Classifying fit into:

- Tight Fit
- Perfect Fit
- Loose Fit

Recommendation Logic

The system uses a **hybrid approach**:

III. **Rule-based mapping** for size conversion

IV. **Data-driven logic** for category recommendations Inputs considered:

V. Extracted measurements

VI. **User height (for proportional accuracy)**

VII. Gender and preferences The system then generates:

VIII. Personalized footwear size

IX. Suggested footwear categories

X. Product recommendations (from dataset)

F. Output Visualization Module

This module presents results in a **clear, structured, and visually appealing format**.

It displays:

I. Foot measurement (in cm)

II. Recommended shoe sizes (UK/US/EU)

III. Fit classification

IV. Suggested product categories and images

UI elements such as **cards, panels, and summary sections** are used to improve readability and user understanding. This enhances overall user satisfaction and trust in the system.

G. Backend and API Layer (Flask)

The backend is implemented using **Flask**, which acts as the central controller of the system.
Its responsibilities include:

I. Handling API requests from frontend

II. Managing image uploads

III. Executing the processing pipeline

IV. Returning structured JSON responses

The backend follows a **modular architecture**, separating:

V. Image processing logic

VI. Recommendation engine

VII. API routing

This ensures scalability, maintainability, and efficient orchestration of all components.

H. Data and Storage Layer

The system uses a flexible storage approach:

I. File system storage for images

II. Structured datasets for footwear categories

Data is organized in a **directory-based structure**, enabling:

III. Efficient retrieval

IV. Easy scalability

V. Future integration with databases (e.g., MongoDB, SQL)

Algorithms And Models Implemented

1. Computer Vision Algorithms (OpenCV – Feature Extraction)

The system uses **OpenCV-based image processing techniques** to extract meaningful visual features from the input image. These algorithms convert raw, unstructured images into structured visual data that can be used for **body measurement estimation**.

Canny Edge Detection

Canny Edge Detection is used to identify **sharp intensity changes** in the image. It detects the boundaries of objects by highlighting edges where pixel intensity changes significantly. In this system, it is used to extract the **clear outline of the human body**, which is essential for understanding body shape and structure.

Contour Detection

Contour detection is used to identify the **continuous boundary of the human body region**. It extracts the outer silhouette of the person from the image and helps in determining the **shape and structure of the body**. These contours are later used to estimate dimensions such as height proportion and body width.

K-Means Clustering (Image Segmentation)

K-Means clustering is used for **image segmentation**, where the image is divided into multiple clusters based on pixel similarity. This helps in separating the **foreground (human body)** from the **background noise**. By isolating the body region, the system improves the accuracy of all subsequent processing steps.

Final Result: The raw image is converted into a **structured body representation with clear boundaries and segmented regions**.

2. Body Measurement Extraction (Pixel to Real-World Conversion)

After feature extraction, the system calculates body measurements from image data and converts them into real-world units.

Pixel-Based Measurement Extraction

The system uses **bounding boxes and contour dimensions** to calculate measurements in pixel form. These pixel values represent the relative dimensions of the body within the image.

Height-Based Scaling (Critical Component)

A key feature of the system is the use of **user-provided height input**, which acts as a reference for scaling. The pixel measurements are converted into real-world units (centimetres) using

proportional scaling based on height. This significantly improves the **accuracy and personalization** of measurement estimation.

Final Result: Accurate estimation of **real-world body measurements in centimetres**, derived from image data and height scaling.

3. Size Mapping Algorithm (Nearest Neighbour Approach)

Once body measurements are extracted, the system maps them to standard clothing sizes using a **Nearest Neighbour-based matching approach**.

Nearest Neighbour Matching

This algorithm compares the user's extracted measurements with predefined **standard size chart values**. It calculates the difference between user measurements and dataset values, and selects the **closest matching size with minimum error distance**.

Size Chart Mapping

The system uses standardized clothing size categories such as **S, M, L, XL or numerical sizing systems** depending on dataset structure. This ensures compatibility with different clothing brands and size standards.

Final Result: The system outputs the **most suitable clothing size based on closest match with standard charts**.

4. Machine Learning Models Used

The system uses multiple **traditional machine learning models** to improve prediction accuracy and handle variability in body measurements.

Linear Regression

Linear Regression is used to treat clothing size prediction as a **continuous numerical problem**. It learns the relationship between **body measurements and size values**, producing a continuous output which is later mapped to the nearest size category.

Strength:

- Captures **linear relationships** between body measurements and clothing sizes • Useful for estimating trend-based size progression

Decision Tree Classifier

The Decision Tree model uses **rule-based splitting logic** to classify clothing sizes. It divides data based on **threshold conditions** such as measurement ranges.

Strength:

- Easy to interpret and explain
- Works well with structured measurement data
- Provides fast classification

Random Forest Classifier

Random Forest is an **ensemble learning model** that combines multiple decision trees. Each tree contributes a vote, and the final output is based on majority decision or averaging.

Strength:

- Reduces **overfitting**
- Handles **non-linear relationships** effectively
- Improves overall prediction accuracy

Ensemble Prediction Strategy

The system combines predictions from **Linear Regression, Decision Tree, and Random Forest models** to generate a final optimized output.

This ensemble approach ensures:

- Higher **accuracy**
- Better **stability**
- Reduced **individual model errors**

Final Result: A more reliable and consistent **clothing size prediction**

5. Rule-Based Logic (Size Recommendation Engine)

In addition to machine learning models, the system uses **rule-based decision logic** for final recommendation refinement.

Size Chart Mapping

The system maps extracted measurements to predefined **industry-standard clothing size charts**.

This ensures consistency with real-world sizing systems used by brands.

Fit Classification

The system classifies the final fit into three categories:

- **Tight Fit**
- **Perfect Fit**
- **Loose Fit**

This helps users understand how well the clothing will fit their body type.

Final Result: Clear and interpretable **fit-based recommendation output**

3.5 Proposed Model

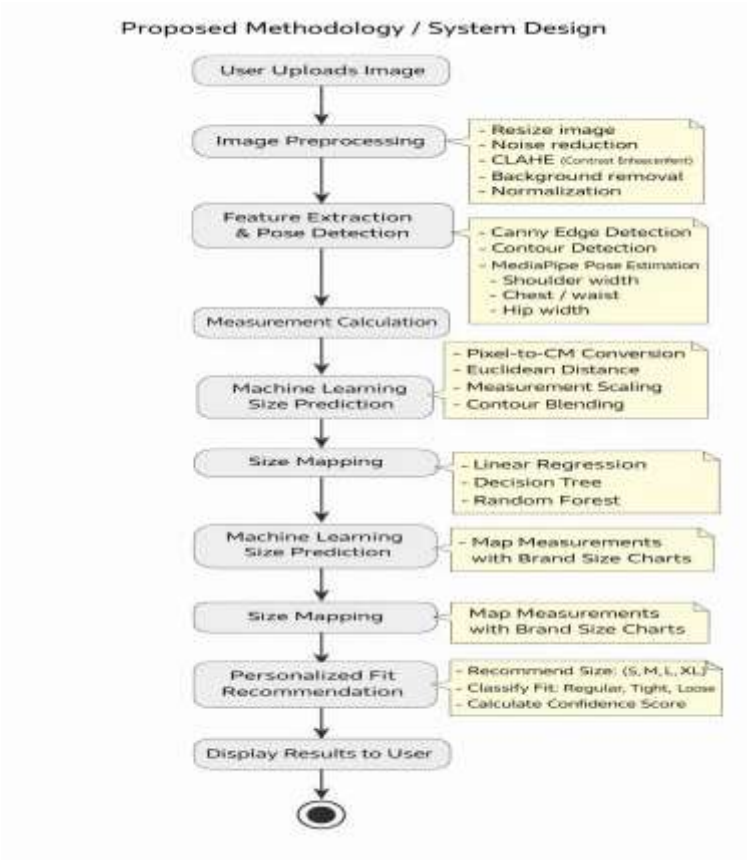


Fig 3.5: Flow chart

Description

- The flowchart represents the step-by-step working of the system from image input to final size recommendation.
- The process begins with the user uploading an image through the interface.
- The image is passed to the preprocessing stage, where **resizing, Gaussian Blur, CLAHE, and background removal (rembg)** are applied.
- The system then performs **pose estimation using MediaPipe Pose** to detect body landmarks.
- Simultaneously, **Canny Edge Detection and Contour Detection** are used to extract the body outline.
- Using these inputs, body measurements are calculated through **Euclidean distance and scaling techniques**.
- The extracted measurements are normalized using **Standard Scaler**.
- The normalized data is given to multiple **Machine Learning models (Linear Regression, Decision Tree, Random Forest)** for size prediction.
- The outputs are combined using an **ensemble approach** to obtain a final prediction.
- The predicted size is mapped to **brand-specific size charts**.
- The final recommended size is displayed to the user.

3.5.1 Dataset Details

The dataset is a fundamental component of the proposed **AI-based Clothing Fit Recommendation System**, as it is used for training and evaluating machine learning models that predict accurate clothing sizes. It contains structured information related to **user body measurements, clothing categories, and corresponding size labels**, enabling the system to learn the relationship between human body proportions and suitable clothing fit. The dataset is designed in a tabular format where each row represents a unique user profile.

The dataset consists of **15 attributes in total**, which include raw body measurements, categorical features, target labels, and normalized features used for improving machine learning performance.

Source of Dataset

The dataset used in this project is prepared using a combination of:

- Publicly available **fashion and apparel datasets (Kaggle sources)**
- Research-based datasets on **clothing size prediction and body measurement systems**
- **Synthetic dataset generation**, created to simulate realistic user body measurements These sources ensure that the dataset contains **diverse body types, measurement variations, and clothing categories**, which are essential for building a robust recommendation system.

Dataset Overview

- **Total Records:** ~5000 user samples
- **Total Features:** 15 attributes
- **File Format:** CSV (Comma Separated Values)
- **Target Variable:** size (S, M, L, XL, etc.)

Each record represents a user with body measurements and their corresponding clothing size.

3.5.1.1 Features (Attributes – 15 Variables)

The dataset contains **15 structured attributes**, which are categorized into raw features, categorical features, target labels, and normalized features.

A. Raw User Measurement Features (6 Attributes)

These features represent actual physical body measurements used for size prediction.

1. **user_id** ○ Unique identifier assigned to each user ○ Used for tracking records and indexing
2. **height** ○ Height of the user in **centimetres (cm)**
   - Important for **scaling and proportional measurement estimation**
3. **chest** ○ Chest circumference measurement ○ Helps determine upper body fit
4. **waist** ○ Waist measurement of the user
   - Critical for **bottom wear and overall fit analysis**
5. **hips** ○ Hip measurement of the user
   - Important for dresses, bottoms, and fitted clothing
6. **shoulder_width** ○ Width of shoulders
   - Used for shirt, tops, and outerwear fitting
7. **inseam** ○ Inner leg length measurement ○ Used mainly for trousers and bottoms

B. Categorical Feature (1 Attribute)

8. **product_details** ○ Represents clothing category
   - Examples: **tops, bottoms, dresses, outerwear**
   - Helps in category-specific size prediction

C. Target Variable (1 Attribute)

9. **size** ○ Final clothing size label
   - Categories: **S, M, L, XL (or numeric equivalents)** ○ This is the **output variable used for prediction**
   - ○

D. Normalized Features (Z-Score Standardized – 6 Attributes)

These features are derived from raw measurements using **Z-score normalization**, which improves machine learning performance by standardizing feature scales.

10. **height_z**
- Normalized height value

11. **chest_z**
- Normalized chest measurement

12. **waist_z**
- Normalized waist measurement

13. **hips_z**
- Normalized hip measurement

14. **shoulder_width_z**
- Normalized shoulder width

15. **inseam_z**
- Normalized inseam value

These normalized features ensure:

- Equal weightage to all features
- Faster model convergence
- Improved prediction accuracy

3.5.1.2 Data Preprocessing

Data Cleaning is performed as the initial step of preprocessing to ensure the dataset is free from missing values, incorrect entries, and noisy data. In this stage, inconsistent records are identified and removed or corrected to maintain data accuracy and reliability. This process

improves the overall quality of the dataset and ensures that the machine learning models are trained on clean and meaningful data.

Feature Standardization is applied to bring all numerical attributes to a common scale. Since the dataset contains multiple body measurements such as height, chest, waist, hips, and inseam, the values vary significantly in range. To handle this, **Z-score normalization** is used to transform the data into a standardized form with zero mean and unit variance. This ensures that all features contribute equally during model training and prevents bias toward larger numerical values.

Encoding of categorical data is performed to convert non-numeric attributes into machine readable format. The feature *product_details*, which includes clothing categories such as tops, bottoms, dresses, and outerwear, is transformed into numerical values. This allows machine learning algorithms to process categorical information effectively and incorporate it into prediction models.

Data Splitting is carried out to divide the dataset into training and testing subsets. The training set is used to train the machine learning models, while the testing set is used to evaluate model performance. This ensures proper validation of the system and helps in preventing overfitting, thereby improving generalization to unseen data.

Feature Preparation is the final step where all processed attributes, including both raw and normalized features, are combined into a structured format suitable for model input. This ensures that the dataset is consistent, well-organized, and optimized for machine learning algorithms, ultimately improving prediction accuracy and system performance.

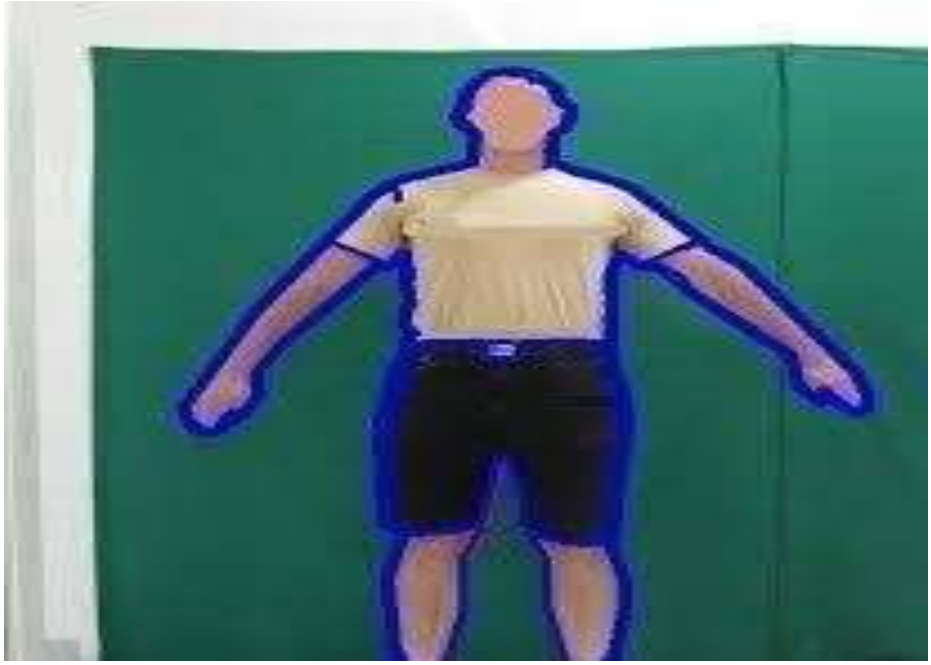


Fig 3.5.1.3 :Business flow diagram

(a) Initial image (b) Traditional Canny (c) Text method

Fig 3.5.1.4: canny Edge detection

4. IMPLEMENTATION AND TESTING

4.1 Implementation Approach

The implementation of the **AI-Driven Personalized Fit Recommendation System** is carried out using a structured and modular approach. The system integrates **computer vision techniques, machine learning models, and a web-based user interface** to generate accurate clothing size recommendations for users.

The frontend of the system allows users to upload their body image or capture a photo using a webcam. The uploaded image is sent to the backend server where image preprocessing and measurement extraction are performed.

The system uses **OpenCV for image processing** to detect body edges and contours. From the processed image, important body measurements such as **height, chest, waist, hips, shoulder width, and inseam** are extracted.

These measurements are then passed to the **machine learning module**, which predicts the most appropriate clothing size using trained models.

Finally, the predicted size is mapped to **brand-specific clothing size charts** stored in the database, and the recommended size is displayed to the user through the web interface.

The implementation of the system follows the steps below:

a) Development of frontend web interface
b) Image preprocessing using OpenCV
c) Body measurement extraction
d) Machine learning model training and prediction
e) Size recommendation using brand size charts
f) Integration of frontend, backend, and ML modules
g) Testing and validation

4.2 Functional and Algorithmic Implementation

This section explains the main algorithms used in the system for image processing, measurement extraction, and clothing size prediction.

4.2.1 Image Preprocessing Algorithm

Input: User body image

Process:

i. Resize image to standard resolution
ii. Convert image to grayscale
iii. Apply noise reduction filters
iv. Normalize brightness and contrast

Output: Preprocessed image suitable for feature extraction

4.2.2 Edge Detection Algorithm

Input: Preprocessed image

Process:

i. Apply **Canny Edge Detection** using OpenCV
ii. Detect body edges from the image
iii. Remove background noise
iv. Highlight body outline

Output: Body edge map

4.2.3 Contour Detection Algorithm

Input: Edge detected image

Process:

i. Identify body contours
ii. Detect continuous boundary of the human body
iii. Extract key measurement points

Output: Body contour representation

4.2.4 Body Measurement Extraction Algorithm

Input: Body contour

Process:

i. Identify measurement reference points
ii. Calculate distances between points
iii. Convert pixel measurements to real body measurements
iv. Store measurement values

Output:

- Height
- Chest
- Waist
- Hips
- Shoulder Width
- Inseam

4.2.5 Size Prediction Algorithm

Input: Body measurements

Process:

i. Load trained machine learning models
ii. Provide measurements as input features
iii. Predict clothing size using models
iv. Generate ensemble prediction

Output: Predicted clothing size

4.2.6 Size Mapping Algorithm

Input: Predicted clothing size

Process:

i. Retrieve brand size chart from database
ii. Compare predicted measurements with chart values
iii. Identify closest matching size

Output: Recommended clothing size

4.3 Integration Details

The system integrates multiple components including **frontend application, backend server, image processing module, machine learning models, and database storage**.

4.3.1 Image Processing Module

The image processing module is implemented using **Python and OpenCV**. It performs operations such as image resizing, edge detection, contour detection, and body measurement extraction.

Code Snippet – Image Preprocessing

```
import cv2

image = cv2.imread("input.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5),0)

edges = cv2.Canny(blur,50,150)
```

This code reads the image, converts it to grayscale, applies Gaussian blur, and performs edge detection.

4.3.2 Feature Extraction Module

The system detects body contours and extracts measurement features using OpenCV functions.

Code Snippet – Contour Detection

```
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

for contour in contours:

    x,y,w,h = cv2.boundingRect(contour)
```

This code detects contours from the edge detected image and identifies bounding rectangles for body measurements.

4.3.3 Machine Learning Prediction Module

The machine learning module predicts clothing size using multiple models.

Code Snippet – Model Training

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

This code trains a Random Forest model to predict clothing sizes.

4.3.4 Backend API Logic

The backend server handles requests from the frontend application and performs image processing and prediction tasks.

Code Snippet – API Endpoint

```
@app.route('/predict', methods=['POST'])
def predict():
    image = request.files['image']
    measurements = extract_measurements(image)
    size = model.predict([measurements])
    return jsonify({"recommended_size": size})
```

This API receives the user image, extracts body measurements, and returns the recommended clothing size.

4.4 Testing

Testing was performed to ensure that all modules of the system function correctly.

The testing includes:

- Unit testing
- Integration testing
- Functional testing

Each module such as image preprocessing, measurement extraction, machine learning prediction, and size recommendation was tested individually before integrating the entire system.

The results show that the system successfully extracts body measurements and predicts suitable clothing sizes.

4.4.1 Test Cases

Different test cases are designed to evaluate system performance in real-world scenarios. When a valid image is uploaded, the system successfully extracts measurements and generates size recommendations. In cases where the image quality is low, the system prompts the user to upload a better image. If the user posture is incorrect, the system may produce less accurate results, but it is designed to handle such situations gracefully.

The system is also tested with extreme body measurements to ensure that it returns the closest valid size. Additionally, scenarios involving missing or invalid inputs are handled by displaying

appropriate error messages. These test cases ensure that the system is **robust, reliable, and user-friendly**.

TEST CASES:

S.No	Test Scenario	Input	Expected Output	Actual Output	Status
1	Upload body image	Clear image	Measurements extracted	Extracted correctly	Pass
2	Low quality image	Blurred image	Warning message	Warning shown	Pass
3	Upload nonbody image	Cake image	Warning message	Warning shown	Pass
5	Upload back side body image	Body image from back side	Measurements extracted	Extracted correctly	Pass

Table 4.4.1.1: Test Cases

5. DISCUSSION OF RESULTS

5.1 Result Analysis

The result analysis of the proposed AI-Driven Personalized Fit Recommendation System evaluates how effectively the system predicts clothing sizes using body measurements extracted from images. The system processes user images, extracts body measurements using computer vision techniques, and predicts suitable clothing sizes using machine learning models.

The system uses multiple machine learning models such as Linear Regression, Random Forest, and Decision Tree to analyze the relationship between body measurements and clothing sizes. These models are trained using a dataset containing body measurement values and corresponding clothing size labels.

Metrics	Linear Regression	Random Forest	Decision Tree
Accuracy	0.98	0.938	0.86
Precision	0.984	0.9332	0.8589
Recall	0.98	0.938	0.86
F1 Score	0.9811	0.93	0.8591

Table 5.1: Accuracy Metrics

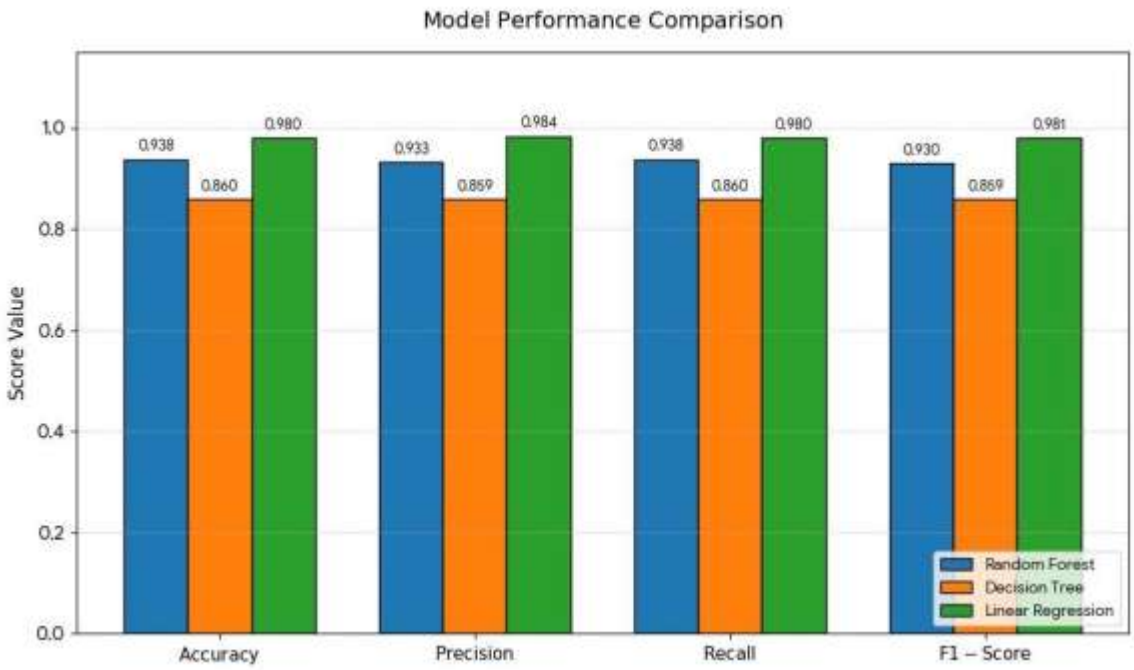


Fig 5.5: Comparison of Model performance

CONCLUSION

6.1 Conclusion

The proposed **AI-Driven Personalized Fit Recommendation System Using Computer Vision** demonstrates how artificial intelligence can be used to improve clothing size selection in online shopping platforms.

The system extracts body measurements from user images using computer vision techniques such as **edge detection and contour detection implemented with OpenCV**. These measurements are then used by machine learning models to predict the most suitable clothing size.

The results show that the system can accurately analyze body measurements and provide reliable clothing size recommendations. It can also detect invalid inputs and display warning messages to avoid incorrect predictions.

By combining **computer vision and machine learning**, the system provides an automated and intelligent solution for clothing size recommendation. This helps reduce size mismatch problems and improves the overall user experience in online fashion shopping.

6.2 FUTURE SCOPE

The system can be further improved by adding advanced technologies and new features.

One possible improvement is the use of **deep learning-based pose estimation models** to improve body measurement accuracy. These models can detect body landmarks more precisely.

Another enhancement is the use of **3D body measurement techniques**, which can provide more accurate body dimensions for size prediction.

The system can also be extended to support **more clothing brands and international size standards**, making it useful for a wider range of users.

In addition, developing a **mobile application** can make the system more accessible by allowing users to capture images directly from their smartphones.

Future versions of the system can also include **user feedback and purchase history** to improve recommendation accuracy and provide more personalized clothing suggestions.

REFERENCES

[1] Rahman M, et al. Toward Smart IoT for Apparel Retail Industry: Automatic Customer's Body Measurements and Size Recommendation System using Computer Vision. IEEE Access. 2021.

[2] Kim S, Moon H, Oh J, Lee Y, Kwon H, Kim S. Automatic Measurements of Garment Sizes Using Computer Vision Deep Learning Models and Point Cloud Data. Applied Sciences. 2022;12(10):5286. doi:10.3390/app12105286

[3] Thota KS Prabhu, Suh S, Zhou B, Lukowicz P. Estimating 3D Body Shape and Clothing Measurements from Frontal and Side-View Images [preprint]. arXiv:2205.14347; 2022.

[4] Alhassawi R, Gill S, Hayes S, Brubacher K. Evaluating Machine Learning Models for Clothing Size Prediction Using Anthropometric Measurements from 3D Body Scanning. Scientific Reports. 2023.

[5] Jogi SS, Khan B, Jamali S, Ali Z, ur Rehman A, Naseebullah, Ali S. AI-Powered Body Type Analysis for Fashion Recommendation System. International Journal of Innovative Science and Technology (IJIST). 2023.

[6] Chen X, et al. A Focused Human Body Model for Accurate Anthropometric Measurements Extraction using CNN-ResNet. In: Proceedings of IEEE CVPR; 2025.

[7] Zhao Y, et al. MeasureXpert: Automatic Anthropometric Measurement Extraction from Two Unregistered Partial Scans. In: Proceedings of IEEE ICCV; 2025.

[8] Li Z, et al. ETCH: Generalizing Body Fitting to Clothed Humans via Equivariant Tightness [preprint]. arXiv; 2025.

[9] Meyer P, Birregah B, Beauseroy P, Grall E, Lauxerrois A. Missing Body Measurements Prediction in Fashion Industry: A Comparative Approach. hion and Textiles. 2023;10:37.

[10] Shirkhani S, Mokayed H, Saini R, Chai HY. Study of AI-Driven Fashion Recommender Systems. SN Computer Science. 2023;4:514.

[11] Kachbal I, El Abdellaoui S. Computer Vision for Fashion: A Systematic Review of Design Generation, Simulation, and Personalized Recommendations. Information. 2026;17(1):11.

[12] Jameel A, Kousar T. Systematic Review of AI-Based Approaches for Anthropometric and Fashion Landmark Detection in Body Measurement Estimation.Journal of Computing & Biomedical Informatics. 2025.

[13] Bag VV, Patil MB, Gaikwad VD, Mohare R, Abhang SP, Antad S. Revolutionizing Fashion: Deep Convolutional Neural Network for Outfit Recommendations. International Journal of Intelligent Systems and Applications in Engineering. 2024.

APPENDEX 1 : CODE

BACKEND:

process_controller.py from __future__ import annotations import cv2

from image_processing.feature_extraction import analyze_features

from image_processing.measurement import calculate_measurements

from image_processing.pose_estimation import

detect_pose_landmarks from image_processing.preprocessing import

preprocess_image from models.measurement_model import

MeasurementModel

def process_image_pipeline(filepath: str, user_id: str, user_height_cm: float | None = None) -> dict: if user_height_cm is not None:

try:

user_height_cm = float(user_height_cm) except

(ValueError, TypeError):

user_height_cm = None preprocessed =

preprocess_image(filepath) image =

preprocessed["foreground"] features =

analyze_features(image) pose =

detect_pose_landmarks(image, min_visibility=0.7)

landmarks = pose["landmarks"] if not landmarks:

raise ValueError("Human not detected in image. Please ensure you are visible and standing straight in the photo.")

measurements = calculate_measurements(

landmarks=landmarks,

contour_metrics=features["contour_metrics"],

```
image_width=image.shape[1], image_height=image.shape[0],
user_height_cm=user_height_cm,
)
method = "mediapipe" warning =
preprocessed["quality"].get("warning") if
pose["mean_visibility"] < 0.5:
warning = (warning + " " if warning else "") + "Please re-upload image standing straight."
method = "opencv" elif pose["source"] == "mediapipe_low_confidence":
method = "ensemble" measurement_id =
MeasurementModel.create(
{
"user_id": user_id,
**measurements,
"image_path": filepath,
"mediapipe_landmarks": {"source": pose["source"], "landmarks": landmarks},
"opencv_contour_data": {"metrics": features["contour_metrics"], "line_count":
len(features["lines"])},
"measurement_method": method,
}
)
return {
"measurement_id": measurement_id,
"measurements": measurements,
"quality": preprocessed["quality"],
"feature_summary": {
"edge_pixels": features["edge_pixels"],
"line_count": len(features["lines"]),
"contours": features["contour_metrics"]["count"],
```

},

"pose_summary": {

"source": pose["source"],

"mean_visibility": pose["mean_visibility"],

"landmark_count": len(landmarks),

},

"warning": warning,

}

recommend_controller.py from __future__ import annotations from collections import Counter from models.brand_model import BrandModel from models.product_model import ProductModel from models.recommendation_model import RecommendationModel from models.user_model import UserModel from ml_model.predict import predict_all_models from recommendation_engine.size_mapper import ensemble_recommendation from utils.converters import SIZE_LABELS def _product_payload(items: list[dict]) -> list[dict]:

payload = [] for

row in items:

payload.append(

{

"product_name": row.get("product_name"),

"product_type": row.get("product_type"),

"image_url": row.get("image_url"),

"product_url": row.get("product_url"),

"source": row.get("source"),

"available_sizes": row.get("available_sizes", []),

}

)

return payload def

_size_index(size: str) -> int:

try:

return SIZE_LABELS.index(size) except

ValueError:

return SIZE_LABELS.index("M") def _apply_history_adjustment(ensemble:

dict, history_rows: list[dict]) -> dict: if not history_rows or len(history_rows)

< 5:

return ensemble

shirt_hist = [row.get("shirt_size") for row in history_rows if row.get("shirt_size") in SIZE_LABELS]

pant_hist = [row.get("pant_size") for row in history_rows if row.get("pant_size") in SIZE_LABELS]

if not shirt_hist and not pant_hist:

return ensemble shirt_mode = Counter(shirt_hist).most_common(1)[0][0] if

shirt_hist else None pant_mode = Counter(pant_hist).most_common(1)[0][0] if

pant_hist else None confidence = float(ensemble.get("confidence_score", 0.0))

shirt_support = (Counter(shirt_hist).most_common(1)[0][1] / len(shirt_hist)) if shirt_hist else 0.0

pant_support = (Counter(pant_hist).most_common(1)[0][1] / len(pant_hist)) if pant_hist else 0.0

changed = [] if shirt_mode and shirt_support >= 0.75 and 45 <= confidence <= 70:

current = ensemble.get("shirt_size") if current in SIZE_LABELS and

abs(_size_index(current) - _size_index(shirt_mode)) <= 1: if current != shirt_mode:

ensemble["shirt_size"] = shirt_mode

ensemble["final_recommendation"] = shirt_mode

changed.append("shirt") if pant_mode and pant_support >= 0.75 and

45 <= confidence <= 70:

```
current = ensemble.get("pant_size") if current in SIZE_LABELS and
abs(_size_index(current) - _size_index(pant_mode)) <= 1: if current != pant_mode:
ensemble["pant_size"] = pant_mode changed.append("pant") if changed:
ensemble["algorithm_results"]["history_adjustment"] = ",".join(changed)
ensemble["confidence_score"] = min(100.0, round(confidence + 4.0, 2))
return ensemble def _recommend_for_brand( measurements: dict,
brand_name: str, user_id: str, measurement_id: str | None = None,
user_profile: dict | None = None, history_by_brand: dict | None = None,
) -> dict:
brand = BrandModel.get_by_name(brand_name)
if not brand:
available = BrandModel.list_brand_names() return {"brand": brand_name, "error":
"Brand not found", "available_brands": available} prefs = (user_profile or
{}).get("preferences", {}) if user_profile else {} fit_pref = str(prefs.get("fit_preference",
"regular")).lower().strip() if fit_pref not in {"slim", "regular", "relaxed"}:
fit_pref = "regular" preferred_brands =
set((prefs.get("preferred_brands") or [])) ml_results =
predict_all_models(measurements) top_chart =
brand.get("size_chart_tops") or brand["size_chart"] bottom_chart =
brand.get("size_chart_bottoms") or brand["size_chart"] ensemble =
ensemble_recommendation( measurements, brand["size_chart"],
ml_results, size_chart_tops=top_chart,
size_chart_bottoms=bottom_chart, fit_preference=fit_pref,
measurement_confidence=measurements.get("confidence_scores", {}),
)
```

```
brand_history = (history_by_brand or {}).get(brand_name, [])
ensemble = _apply_history_adjustment(ensemble, brand_history) if
brand_name in preferred_brands:
ensemble["confidence_score"] = min(100.0, round(float(ensemble["confidence_score"]) + 3.5, 2))
ensemble["algorithm_results"]["preference_adjustment"] = "preferred_brand_boost"
shirt_products = ProductModel.get_recommendations( brand_name,
"shirts", ensemble["shirt_size"],
fit_type=ensemble.get("shirt_fit_type", "regular"),
limit=3,
)
pant_products = ProductModel.get_recommendations(
brand_name, "pants", ensemble["pant_size"],
fit_type=ensemble.get("pant_fit_type", "regular"),
limit=3,
)
RecommendationModel.create(
{
"user_id": user_id,
"measurement_id": measurement_id,
"brand": brand_name,
"recommended_size": ensemble["final_recommendation"],
"fit_type": ensemble["fit_type"],
"shirt_size": ensemble["shirt_size"],
"pant_size": ensemble["pant_size"],
"shirt_fit_type": ensemble["shirt_fit_type"],
"pant_fit_type": ensemble["pant_fit_type"],
"algorithm_results": ensemble["algorithm_results"], "confidence_score":
ensemble["confidence_score"],
```

```
"size_chart_used": {
"top_chart": top_chart,
"bottom_chart": bottom_chart,
},
}
)
return {
"brand": brand_name,
"final_recommendation": ensemble["final_recommendation"],
"fit_type": ensemble["fit_type"],
"confidence_score": ensemble["confidence_score"],
"algorithm_results": ensemble["algorithm_results"],
"shirt_recommendation": {
"size": ensemble["shirt_size"],
"fit_type": ensemble["shirt_fit_type"],
"algorithm_results": ensemble["algorithm_results_shirt"],
"products": _product_payload(shirt_products),
},
"pant_recommendation": {
"size": ensemble["pant_size"],
"fit_type": ensemble["pant_fit_type"],
"algorithm_results": ensemble["algorithm_results_pant"],
"products": _product_payload(pant_products),
},
"product_recommendations": {
"shirts": _product_payload(shirt_products),
"pants": _product_payload(pant_products),
```

```
},
}


def recommend_sizes(payload: dict) -> dict:
measurements = payload["measurements"] user_id
= payload["user_id"] measurement_id =
payload.get("measurement_id") brands =
payload.get("brands") if not brands: if
payload.get("brand"): brands = [payload["brand"]]
else:
brands = BrandModel.list_brand_names() user_profile =
UserModel.get_by_id(user_id) prior =
RecommendationModel.get_by_user_id(user_id, limit=60)
history_by_brand: dict[str, list[dict]] = {} for row in prior:
history_by_brand.setdefault(row.get("brand"), []).append(row)

recommendations = [
_recommend_for_brand(
measurements=measurements,
brand_name=brand_name, user_id=user_id,
measurement_id=measurement_id,
user_profile=user_profile,
history_by_brand=history_by_brand,
)
for brand_name in brands
]
```

if len(recommendations) == 1: return

recommendations[0] return

{"recommendations": recommendations}

upload_controller.py from __future__ import annotations import

base64filepath": filepathhlib import Path from uuid import uuid4 import

cv2 import numpy as np from flask import current_app from PIL import

Image from werkzeug.datastructures import FileStorage from

werkzeug.utils import secure_filename from utils.validators import

allowed_file, validate_image_quality def _save_image_array(image:

np.ndarray) -> str: upload_dir =

Path(current_app.config["UPLOAD_DIR"])

upload_dir.mkdir(parents=True, exist_ok=True) filename =

f"{uuid4().hex}.jpg" path = upload_dir / filename cv2.imwrite(str(path),

image, [int(cv2.IMWRITE_JPEG_QUALITY), 85])

return str(path)

def _decode_image_bytes(raw_bytes: bytes) -> np.ndarray | None:

arr = np.frombuffer(raw_bytes, np.uint8) image =

cv2.imdecode(arr, cv2.IMREAD_COLOR)

if image is not None:

return image

Fallback for formats OpenCV may not decode (e.g., some AVIF builds).

try:

```
pil_img = Image.open(io.BytesIO(raw_bytes)).convert("RGB")
rgb = np.array(pil_img) return cv2.cvtColor(rgb,
cv2.COLOR_RGB2BGR) except Exception:  # noqa: BLE001
return None


def handle_upload(file: FileStorage) -> dict: if
not file:
raise ValueError("No file provided.") if
not allowed_file(file.filename):
raise ValueError("Unsupported file format.")
filename = secure_filename(file.filename)
raw_bytes = file.read() image =
_decode_image_bytes(raw_bytes) if image
is None:
raise ValueError("Unsupported or corrupted image data.")
quality = validate_image_quality(image) filepath =
_save_image_array(image) return {"filename": filename, "filepath":
filepath, "quality": quality}


def handle_webcam_upload(base64_image: str) -> dict: if
"," in base64_image:
base64_image = base64_image.split(",", 1)[1]
image_bytes = base64.b64decode(base64_image) arr
= np.frombuffer(image_bytes, dtype=np.uint8)
```

image = cv2.imdecode(arr, cv2.IMREAD_COLOR)

if image is None:

raise ValueError("Invalid webcam image payload.")

quality = validate_image_quality(image) filepath =

_save_image_array(image) return {"filepath":

filepath, "quality": quality}

user_controller.py

from __future__ import annotations from pathlib import Path from

flask import request from models.measurement_model import

MeasurementModel from models.recommendation_model import

RecommendationModel from models.user_model import UserModel

def create_user(payload: dict) -> dict: if "name" not in

payload or "email" not in payload: raise

ValueError("Both name and email are required.")

user_id = UserModel.create(payload) return

{"user_id": user_id} def list_users() -> list[dict]:

return UserModel.get_all() def get_user(user_id: str) -

> dict | None: return UserModel.get_by_id(user_id)

def delete_user(user_id: str) -> bool: return

UserModel.delete(user_id)

def user_recommendation_history(user_id: str, limit: int | None = None, grouped: bool = False) > list[dict]:

history = RecommendationModel.get_by_user_id(user_id, limit=limit) measurements =

MeasurementModel.get_by_user_id(user_id) measurements = sorted(measurements,

```
key=lambda x: x.get("created_at"), reverse=True) def _fallback_measurement(rec: dict)
-> dict | None: if not measurements:
return None rec_time = rec.get("created_at") if rec_time is None: return measurements[0]
valid = [m for m in measurements if m.get("created_at") and m.get("created_at") <= rec_time]
return valid[0] if valid else measurements[0] for item in history: measurement = None
measurement_id = item.get("measurement_id") if measurement_id:

measurement = MeasurementModel.get_by_id(measurement_id)
if measurement is None: measurement =
_fallback_measurement(item) image_filename = None
image_url = None analysis = None if measurement:
image_path = measurement.get("image_path") if
image_path:
image_filename = Path(image_path).name image_url =
f"{request.url_root.rstrip('/')}/api/uploads/{image_filename}" analysis = {
"measurements": {
"height": measurement.get("height"),
"chest": measurement.get("chest"),
"waist": measurement.get("waist"),
"hips": measurement.get("hips"),
"shoulder_width": measurement.get("shoulder_width"),
"inseam": measurement.get("inseam"),
},
"confidence_scores": measurement.get("confidence_scores", {}),
"measurement_method": measurement.get("measurement_method"),
"measurement_created_at": measurement.get("created_at"),
"pose_source": (measurement.get("mediapipe_landmarks") or {}).get("source"),
"contour_summary": (measurement.get("opencv_contour_data") or {}).get("metrics"), }
```

```
item["image_filename"] = image_filename
item["image_url"] = image_url
item["analysis"] = analysis if not grouped:
return history grouped_map = {}
for item in history:
analysis = item.get("analysis") or {} measurement_created_at =
analysis.get("measurement_created_at")
group_key = item.get("measurement_id") or
f"{item.get('image_filename')}_{measurement_created_at}" if group_key not in grouped_map:
grouped_map[group_key] = { "_id": str(group_key),
"user_id": item.get("user_id"),
"created_at": item.get("created_at"),
"image_filename": item.get("image_filename"),
"image_url": item.get("image_url"),
"analysis": analysis,
"brands": [],
}

grouped_map[group_key]["brands"].append(
{
"recommendation_id": item.get("_id"),
"brand": item.get("brand"),
"recommended_size": item.get("recommended_size"),
"fit_type": item.get("fit_type"),
"shirt_size": item.get("shirt_size"),
"pant_size": item.get("pant_size"),
"shirt_fit_type": item.get("shirt_fit_type"),
```

```
"pant_fit_type": item.get("pant_fit_type"),
"confidence_score": item.get("confidence_score"),
"algorithm_results": item.get("algorithm_results", {}),
}
)
grouped_history = list(grouped_map.values()) for
row in grouped_history:
row["total_brands"] = len(row["brands"]) row["brands"] = sorted(row["brands"],
key=lambda x: x.get("brand") or "") grouped_history = sorted(grouped_history, key=lambda
x: x.get("created_at"), reverse=True) return grouped_history
```

feature_extraction.py from __future__ import

```
annotations import cv2 import numpy as np def
canny_edges(imacolorr: np.ndarray) -> np.ndarray:
gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
return cv2.Canny(gray, threshold1=50, threshold2=150) def
hough_lines(edges: np.ndarray):
return cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=45,
minLineLength=40, maxLineGap=12)

def find_body_contours(image_bgr: np.ndarray) -> tuple[list[np.ndarray], np.ndarray]:
gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY) _,
thresh = cv2.threshold(gray, 1, 255, cv2.THRESH_BINARY)
dilated = cv2.dilate(thresh, np.ones((3, 3), np.uint8), iterations=2)
contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

```
largest = max(contours, key=cv2.contourArea) if contours else None
hull = cv2.convexHull(largest) if largest is not None else None return
contours, hull

def contour_metrics(contours: list[np.ndarray], hull: np.ndarray | None) -> dict: if
not contours:
return {"count": 0, "largest_area": 0.0, "bbox": None, "hull_area": 0.0, "profile_widths_px": {}}
largest = max(contours, key=cv2.contourArea) x, y, w, h = cv2.boundingRect(largest) hull_area
= float(cv2.contourArea(hull)) if hull is not None else 0.0

# Sample silhouette widths at torso-relevant heights for better measurements
local = largest.copy() local[:, 0, 0] -= x local[:, 0, 1] -= y mask =
np.zeros((h, w), dtype=np.uint8) cv2.drawContours(mask, [local],
contourIdx=-1, color=255, thickness=-1)

def width_at_ratio(ratio: float) -> int:
row_idx = int(max(0, min(h - 1, round((h - 1) * ratio))))
row = mask[row_idx] cols = np.where(row > 0)[0]

if cols.size == 0:
return 0 return int(cols[-1] -
cols[0] + 1) profile_widths = {
"chest": width_at_ratio(0.32),
"waist": width_at_ratio(0.48),
"hips": width_at_ratio(0.62),
} return
{
"count": len(contours),
```

"largest_area": float(cv2.contourArea(largest)),

"bbox": {"x": int(x), "y": int(y), "w": int(w), "h": int(h)},

"hull_area": hull_area,

"profile_widths_px": profile_widths,

} def analyze_features(image_bgr: np.ndarray) ->

dict:

edges = canny_edges(image_bgr) lines = hough_lines(edges) contours,

hull = find_body_contours(image_bgr) metrics =

contour_metrics(contours, hull) lines_list = [] if lines is not None: for line

in lines[:200]: x1, y1, x2, y2 = line[0] lines_list.append({"x1": int(x1),

"y1": int(y1), "x2": int(x2), "y2": int(y2)}) return {

"edges": edges,

"edge_pixels": int(np.sum(edges > 0)),

"lines": lines_list,

"contours": contours,

"hull": hull,

"contour_metrics": metrics,

}

measurement.py from __future__

import annotations

impors_x, rs_y, rs_v = _xy_from_landmarkarks: list[dict], index: int, width: int, height: int) -> tuple[float, float, float]: lm = landmarks[index] return lm["x"] * width, lm["y"] * height, lm.get("visibility", 0.0) def _distance(a: tuple[float, float], b: tuple[float, float]) -> float:

return math.dist(a, b) def _clamp(value: float, low:

float, high: float) -> float:

return max(low, min(high, value))

def _estimate_height_cm(

```
full_height_px: float, shoulder_px:
float, hip_px: float, inseam_px: float,
bbox: dict | None, user_height_cm:
float | None = None,
) -> tuple[float, float]: if user_height_cm
and user_height_cm > 0:
return user_height_cm, 1.0
# Height cannot be directly recovered from a monocular image, so we estimate
# it with several anthropometric priors and combine them robustly. if
full_height_px <= 1:
return 170.0, 0.7

bbox_w = float(bbox.get("w", 0)) if bbox else 0.0 bbox_h =
float(bbox.get("h", 0)) if bbox else 0.0 body_aspect = bbox_h /
max(bbox_w, 1.0) if bbox_w > 0 else 2.7

expected_shoulder_cm = _clamp(42.0 + ((2.8 - body_aspect) * 2.8), 35.0, 52.0)
expected_hip_cm = _clamp(0.9 * expected_shoulder_cm + 2.0, 34.0, 50.0) expected_inseam_cm
= _clamp(0.46 * 170.0 + ((body_aspect - 2.6) * 5.0), 68.0, 94.0)

estimates: list[tuple[float, float]] = [] if
shoulder_px > 2:
estimates.append((((full_height_px / shoulder_px) * expected_shoulder_cm), 0.38)) if
hip_px > 2:
estimates.append((((full_height_px / hip_px) * expected_hip_cm), 0.24)) if
inseam_px > 2:
```

```
estimates.append((((full_height_px / inseam_px) * expected_inseam_cm), 0.26)) if
bbox and bbox_w > 2 and bbox_h > 2:
estimates.append((((bbox_h / bbox_w) * 44.0), 0.12))

if not estimates: return
170.0, 0.65

weighted_sum = sum(v * w for v, w in estimates) total_w
= sum(w for _, w in estimates)

height = weighted_sum / max(total_w, 1e-6)
# Only clamp if user height not provided (assume adult for default inference) if
not user_height_cm:
height = _clamp(height, 145.0, 205.0) values = [v for v, _ in estimates] mean_val
= sum(values) / len(values) spread = math.sqrt(sum((v - mean_val) ** 2 for v in
values) / max(len(values), 1)) stability = _clamp(1.0 - (spread / max(mean_val,
1.0)), 0.55, 0.98) return height, stability def calculate_measurements( landmarks:
list[dict], contour_metrics: dict, image_width: int, image_height: int,
user_height_cm: float | None = None, ) -> dict: if not landmarks or
len(landmarks) < 29:
raise ValueError("Insufficient landmarks for measurement extraction.") ls_x, ls_y, ls_v =
_xy_from_landmark(landmarks, 11, image_width, image_height) rs_x, rs_y, rs_v =
_xy_from_landmark(landmarks, 12, image_width, image_height) lh_x, lh_y, lh_v =
_xy_from_landmark(landmarks, 23, image_width, image_height) rh_x, rh_y, rh_v =
_xy_from_landmark(landmarks, 24, image_width, image_height) nose_x, nose_y, nose_v
= _xy_from_landmark(landmarks, 0, image_width, image_height) la_x, la_y, la_v =
_xy_from_landmark(landmarks, 27, image_width, image_height) ra_x, ra_y, ra_v =
_xy_from_landmark(landmarks, 28, image_width, image_height)
```

```
shoulder_px = _distance((ls_x, ls_y), (rs_x, rs_y))
hip_px = _distance((lh_x, lh_y), (rh_x, rh_y)) ankle_mid
= ((la_x + ra_x) / 2.0, (la_y + ra_y) / 2.0) hip_mid =
((lh_x + rh_x) / 2.0, (lh_y + rh_y) / 2.0) full_height_px =
_distance((nose_x, nose_y), ankle_mid) inseam_px =
_distance(hip_mid, ankle_mid)

bbox = contour_metrics.get("bbox") if contour_metrics else None
if full_height_px < 1 and bbox: full_height_px = bbox["h"]

# User-provided height is now handled in _estimate_height_cm.
real_height, height_conf_multiplier = _estimate_height_cm( full_height_px,
shoulder_px, hip_px, inseam_px, bbox, user_height_cm
)

px_to_cm = real_height / max(full_height_px, 1.0) profile =
contour_metrics.get("profile_widths_px", {}) if contour_metrics else {} body_width_px =
float(bbox["w"]) if bbox and bbox.get("w", 0) > 0 else shoulder_px * 2.05 chest_width_px
= float(profile.get("chest") or (body_width_px * 0.78)) waist_width_px =
float(profile.get("waist") or (body_width_px * 0.70)) hips_width_px =
float(profile.get("hips") or (body_width_px * 0.82))

# Scale clamps based on height if it's a child (height < 145cm)
base_h = 170.0 scale_factor = real_height / base_h is_child =
real_height < 145.0
```

```
# Proportion-based measurements scaled by px_to_cm for more accuracy with user height
shoulder_cm = shoulder_px * px_to_cm if not user_height_cm:
# Fallback to ratio-based if no user height (original logic) shoulder_ratio =
shoulder_px / max(full_height_px, 1.0) shoulder_cm = 43.0 * (shoulder_ratio /
0.245) s_min, s_max = (20.0, 65.0) if is_child else (32.0, 58.0) shoulder_cm =
_clamp(shoulder_cm, s_min, s_max) chest_cm = chest_width_px * px_to_cm *
1.57 # Rough width to circumference if not user_height_cm:
chest_ratio = chest_width_px / max(full_height_px, 1.0)
chest_from_width_ratio = 96.0 * (chest_ratio / 0.225) chest_from_shoulder
= shoulder_cm * 2.15 chest_cm = (0.45 * chest_from_width_ratio) + (0.55 *
chest_from_shoulder) c_min = max(45.0 if is_child else 76.0, shoulder_cm
* 1.8) c_max = min(180.0, shoulder_cm * 2.8) chest_cm =
_clamp(chest_cm, c_min, c_max) waist_cm = waist_width_px * px_to_cm
* 1.57 if not user_height_cm:
waist_ratio = waist_width_px / max(full_height_px, 1.0)
waist_from_width_ratio = 82.0 * (waist_ratio / 0.195) waist_from_chest
= chest_cm * 0.845 waist_cm = (0.65 * waist_from_width_ratio) + (0.35
* waist_from_chest)

w_min = max(40.0 if is_child else 58.0, chest_cm * 0.65)
w_max = min(160.0, chest_cm * 1.1) waist_cm =
_clamp(waist_cm, w_min, w_max)

hips_cm = hips_width_px * px_to_cm * 1.57 if
not user_height_cm:
hips_ratio = hips_width_px / max(full_height_px, 1.0) hip_ratio =
hip_px / max(full_height_px, 1.0) hips_from_width_ratio = 98.0 *
```

```
(max(hips_ratio, hip_ratio * 1.2) / 0.24) hips_from_hip_ratio = 98.0 *
(hip_ratio / 0.21) hips_cm = (0.5 * hips_from_width_ratio) + (0.5 *
hips_from_hip_ratio)

h_min = max(45.0 if is_child else waist_cm * 1.03, waist_cm * 0.9)
h_max = min(180.0, chest_cm * 1.3) hips_cm = _clamp(hips_cm,
h_min, h_max)

inseam_cm = inseam_px * px_to_cm if
inseam_cm < (real_height * 0.3):
inseam_cm = real_height * 0.45

i_min, i_max = (30.0, 115.0) if is_child else (60.0, 105.0)
inseam_cm = _clamp(inseam_cm, i_min, i_max) height_cm
= real_height

conf_base = max(0.35, min(0.99, (ls_v + rs_v + lh_v + rh_v + nose_v + la_v + ra_v) / 7.0))
confidences = {
"height": round(conf_base * 0.98 * 100 * height_conf_multiplier, 2),
"chest": round(conf_base * 0.94 * 100, 2),
"waist": round(conf_base * 0.92 * 100, 2),
"hips": round(conf_base * 0.93 * 100, 2),
"shoulder_width": round(conf_base * 0.95 * 100, 2),
"inseam": round(conf_base * 0.90 * 100, 2),
}

return {
```

```
"height": round(height_cm, 2),
"chest": round(chest_cm, 2),
"waist": round(waist_cm, 2),
"hips": round(hips_cm, 2),
"shoulder_width": round(shoulder_cm, 2),
"inseam": round(inseam_cm, 2),
"confidence_scores": confidences,
}
```

pose_estimation.py from

```
__future__ import annotations
import math import numpy as np
try:
import mediapipe as mp except Exception:  # noqa: BLE001 mp = None def
detect_pose_landmarks(image_bgr: np.ndarray, min_visibility: float = 0.7) -> dict:
h, w = image_bgr.shape[:2] if
mp is None:
return {"landmarks": [], "source": "lib_missing", "mean_visibility": 0.0}

rgb = image_bgr[:, :, ::-1] with
mp.solutions.pose.Pose(
static_image_mode=True,
model_complexity=1,
enable_segmentation=False,
min_detection_confidence=0.5,
min_tracking_confidence=0.5,
) as pose: results =
pose.process(rgb)
```

```
if not results.pose_landmarks:
return {"landmarks": [], "source": "no_human_detected", "mean_visibility": 0.0}
landmarks = [] vis = [] for idx, lm in
enumerate(results.pose_landmarks.landmark):
visibility = float(lm.visibility)
vis.append(visibility) landmarks.append(
{
"index": idx,
"x": float(lm.x),
"y": float(lm.y),
"z": float(lm.z),
"visibility": visibility,
}
)
mean_visibility = float(np.mean(vis)) if vis else 0.0
# If confidence is too low, keep data but mark with source for fallback decisions.
source = "mediapipe" if mean_visibility >= min_visibility else "mediapipe_low_confidence"
return {"landmarks": landmarks, "source": source, "mean_visibility": mean_visibility}
```

preprocessing.py

```
from __future__ import annotations import
isinstancethlib import Path import cv2 import
numpy as np from utils.validators import
validate_image_quality
try:
```

```
from rembg import remove except Exception:  # noqa: BLE001 remove = None
def _resize_to_height(image: np.ndarray, target_height: int = 600) -> np.ndarray:
h, w = image.shape[:2] if
h == target_height:
return image ratio = target_height / float(h) target_width = max(1, int(w * ratio)) return
cv2.resize(image, (target_width, target_height), interpolation=cv2.INTER_AREA) def
_apply_clahe(image_bgr: np.ndarray) -> np.ndarray: lab = cv2.cvtColor(image_bgr,
cv2.COLOR_BGR2LAB) l_channel, a_channel, b_channel = cv2.split(lab) clahe =
cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)) cl = clahe.apply(l_channel)

merged = cv2.merge((cl, a_channel, b_channel)) return
cv2.cvtColor(merged, cv2.COLOR_LAB2BGR)

def _hist_equalize_fallback(image_bgr: np.ndarray) -> np.ndarray: ycrcb
= cv2.cvtColor(image_bgr, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(ycrcb) y = cv2.equalizeHist(y)
merged = cv2.merge((y, cr, cb)) return
cv2.cvtColor(merged, cv2.COLOR_YCrCb2BGR)

def _remove_background(image_bgr: np.ndarray) -> np.ndarray:
if remove is None: return image_bgr success, buf =
cv2.imencode(".png", image_bgr) if not success: return
image_bgr fg_bytes = remove(buf.tobytes()) fg_arr =
np.frombuffer(fg_bytes, dtype=np.uint8) decoded =
cv2.imdecode(fg_arr, cv2.IMREAD_UNCHANGED)
if decoded is None: return image_bgr if decoded.shape[-1] ==
4: alpha = decoded[:, :, 3] / 255.0 rgb = decoded[:, :,
:3].astype(np.float32) bg = np.zeros_like(rgb) merged = (rgb *
```

```
alpha[..., None]) + (bg * (1 - alpha[..., None])) return
merged.astype(np.uint8)

return decoded


def preprocess_image(image_input: str | Path | np.ndarray) -> dict: if
isinstance(image_input, (str, Path)):
image_path = str(image_input) raw
= cv2.imread(image_path)
else:
raw = image_input image_path
= None


if raw is None:
raise ValueError("Unable to load image for preprocessing.")


quality = validate_image_quality(raw) resized =
_resize_to_height(raw, target_height=600) blurred
= cv2.GaussianBlur(resized, (5, 5), 0) enhanced =
_apply_clahe(blurred)


# Fallback to histogram equalization when global contrast is too low.
gray = cv2.cvtColor(enhanced, cv2.COLOR_BGR2GRAY)
if float(np.std(gray)) < 20:
enhanced = _hist_equalize_fallback(enhanced)


foreground = _remove_background(enhanced) normalized
= foreground.astype(np.float32) / 255.0
```

```
return {
"image_path": image_path,
"original": raw,
"resized": resized,
"blurred": blurred,
"enhanced": enhanced,
"foreground": foreground,
"normalized": normalized,
"quality": quality,
}
```

evaluate.py from __future__

```
import annotations import json
from pathlib import Path
MODEL_DIR = Path(__file__).resolve().parent / "saved_models" def
load_metrics() -> dict:
metrics_path = MODEL_DIR / "metrics.json" if
not metrics_path.exists():
return {} with open(metrics_path, "r",
encoding="utf-8") as f:
return json.load(f) if __name__ ==
"__main__": metrics =
load_metrics()
print(json.dumps(metrics, indent=2))
```

predict.py

```
from __future__ import annotations

from pathlib import Path import joblib import
numpy as np from utils.convertersdtypert
clamp_size_label
FEATURES = ["height", "chest", "waist", "hips", "shoulder_width", "inseam"]
MODEL_DIR = Path(__file__).resolve().parent / "saved_models"
_CACHE = {}
def _load_models():
if _CACHE: return
_CACHE
paths = {
"random_forest": MODEL_DIR / "random_forest.pkl",
"decision_tree": MODEL_DIR / "decision_tree.pkl",
"linear_regression": MODEL_DIR / "linear_regression.pkl",
"scaler": MODEL_DIR / "scaler.pkl",
"label_encoder": MODEL_DIR / "label_encoder.pkl",
} for key, path in
paths.items(): if not
path.exists():
raise FileNotFoundError(f"Model artifact missing: {path}")
_CACHE.update({k: joblib.load(v) for k, v in paths.items()}) return
_CACHE

def model_files_exist() -> bool:
required = ["random_forest.pkl", "decision_tree.pkl", "linear_regression.pkl", "scaler.pkl",
"label_encoder.pkl"]
```

return all((MODEL_DIR / name).exists() for name in required)

def predict_all_models(measurements: dict) -> dict: if not model_files_exist(): return {"random_forest": "M", "decision_tree": "M", "linear_regression": "M"} models = _load_models() vector = np.array([[float(measurements.get(col, 0.0)) for col in FEATURES]], dtype=float) vector_scaled = models["scaler"].transform(vector) encoder = models["label_encoder"] rf_idx = int(models["random_forest"].predict(vector_scaled)[0]) dt_idx = int(models["decision_tree"].predict(vector_scaled)[0]) lr_raw = float(models["linear_regression"].predict(vector_scaled)[0]) lr_idx = int(np.clip(round(lr_raw), 0, len(encoder.classes_) - 1)) return { "random_forest": clamp_size_label(encoder.inverse_transform([rf_idx])[0], encoder.classes_), "decision_tree": clamp_size_label(encoder.inverse_transform([dt_idx])[0], encoder.classes_), "linear_regression": clamp_size_label(encoder.inverse_transform([lr_idx])[0], encoder.classes_), }

train_model.py from __future__ import annotations import json import os from pathlib import Path import numpy as np import pandas as pd from sklearn.ensemble import RandomForestClassifier from sklearn.linear_model import LinearRegression from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score from sklearn.model_selection import train_test_split from sklearn.preprocessing import LabelEncoder, StandardScaler from sklearn.tree import DecisionTreeClassifier import joblib ROOT = Path(__file__).resolve().parents[2]

```
DATASET_RAW_DIR = ROOT / "dataset" / "raw"
DATASET_PROCESSED_DIR = ROOT / "dataset" / "processed"
MODEL_DIR = Path(__file__).resolve().parent / "saved_models"
FEATURES = ["height", "chest", "waist", "hips", "shoulder_width", "inseam"]
TARGET = "size"
SIZE_LABELS = ["XS", "S", "M", "L", "XL", "XXL"] def
_assign_size(row: pd.Series) -> str:
score = (0.2 * row["chest"]) + (0.2 * row["waist"]) + (0.2 * row["hips"]) + (0.1 * row["shoulder_width"]) + (0.1 * row["inseam"]) + (0.2 * row["height"]) if score < 89: return "XS" if score < 100:
return "S" if
score < 112:
return "M" if
score < 124:
return "L" if
score < 138:
return "XL"
return "XXL"
def
generate_synt
hetic_dataset(
n_rows: int =
5000, seed:
int = 42) ->
pd.DataFrame
:
```

```
rng = np.random.default_rng(seed) height =
rng.normal(170, 10, n_rows).clip(145, 205) chest =
rng.normal(96, 13, n_rows).clip(70, 165) waist =
rng.normal(82, 14, n_rows).clip(50, 150) hips =
rng.normal(97, 12, n_rows).clip(72, 170)
shoulder_width = rng.normal(43, 5, n_rows).clip(30, 60)
inseam = rng.normal(78, 8, n_rows).clip(55, 105)

df = pd.DataFrame(
{
"user_id": [f"user_{i+1:05d}" for i in range(n_rows)],
"height": height.round(2),
"chest": chest.round(2),
"waist": waist.round(2),
"hips": hips.round(2),
"shoulder_width": shoulder_width.round(2),
"inseam": inseam.round(2),
"product_details": rng.choice(["tops", "bottoms", "dresses", "outerwear"], size=n_rows),
}
)
df["size"] = df.apply(_assign_size, axis=1)
return df def load_dataset() ->
pd.DataFrame:
DATASET_PROCESSED_DIR.mkdir(parents=True, exist_ok=True) processed_csv
= DATASET_PROCESSED_DIR / "training_dataset.csv" if processed_csv.exists():

return pd.read_csv(processed_csv)
```

```
# Fallback strategy: generate synthetic data when external dataset is unavailable. df
= generate_synthetic_dataset()

# Remove invalid values and normalize z-score columns for ML support diagnostics.
df = df.replace([np.inf, -np.inf], np.nan).dropna() for col in FEATURES: df = df[df[col] >
0] zscore_cols = [f"{col}_z" for col in FEATURES] df[zscore_cols] = (df[FEATURES] -
df[FEATURES].mean()) / df[FEATURES].std(ddof=0) df.to_csv(processed_csv,
index=False) return df



def _classification_metrics(y_true, y_pred) -> dict:
return {
"accuracy": round(float(accuracy_score(y_true, y_pred)), 4),
"precision": round(float(precision_score(y_true, y_pred, average="weighted", zero_division=0)),
4),
"recall": round(float(recall_score(y_true, y_pred, average="weighted", zero_division=0)), 4),
"f1": round(float(f1_score(y_true, y_pred, average="weighted", zero_division=0)), 4), }



def train_and_save_models() -> dict:
MODEL_DIR.mkdir(parents=True, exist_ok=True)
df = load_dataset()

X = df[FEATURES].copy() y
= df[TARGET].copy()
```

```
encoder = LabelEncoder() y_encoded
= encoder.fit_transform(y) scaler =
StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

rf_model = RandomForestClassifier(n_estimators=300, random_state=42)
dt_model = DecisionTreeClassifier(max_depth=10, random_state=42) lr_model
= LinearRegression()

rf_model.fit(X_train, y_train)
dt_model.fit(X_train, y_train) lr_model.fit(X_train,
y_train)

rf_pred = rf_model.predict(X_test) dt_pred =
dt_model.predict(X_test) lr_pred =
np.rint(lr_model.predict(X_test)).astype(int) lr_pred =
np.clip(lr_pred, 0, len(encoder.classes_) - 1) metrics =
{
"random_forest": _classification_metrics(y_test, rf_pred),
"decision_tree": _classification_metrics(y_test, dt_pred),
"linear_regression": _classification_metrics(y_test, lr_pred),
}
```

```
joblib.dump(rf_model, MODEL_DIR / "random_forest.pkl")
joblib.dump(dt_model, MODEL_DIR / "decision_tree.pkl")
joblib.dump(lr_model, MODEL_DIR / "linear_regression.pkl")
joblib.dump(scaler, MODEL_DIR / "scaler.pkl") joblib.dump(encoder,
MODEL_DIR / "label_encoder.pkl")

with open(MODEL_DIR / "metrics.json", "w", encoding="utf-8") as f:
json.dump(metrics, f, indent=2)

return metrics



if __name__ == "__main__": results
= train_and_save_models()
print("Training complete.")
print(json.dumps(results, indent=2))
```

brand_model.py

```
from __future__ import annotations

from datetime import datetime, timezone from
dcls.get_allconnection import get_db class
BrandModel:
COLLECTION = "brands" @classmethod
def _collection(cls):
col = get_db()[cls.COLLECTION]
try:
```

```
col.create_index([("brand_name", 1)], unique=True)
except Exception:  # noqa: BLE001 pass return col

@classmethod
def upsert( cls,
brand_name: str, country: str, size_chart:
dict, international_standards: dict | None =
None, size_chart_tops: dict | None = None,
size_chart_bottoms: dict | None = None,
):
doc = {
"brand_name": brand_name,
"country": country,
"size_chart": size_chart,
"size_chart_tops": size_chart_tops or size_chart,
"size_chart_bottoms": size_chart_bottoms or size_chart,
"international_standards": international_standards or {},
"updated_at": datetime.now(timezone.utc),
} cls._collection().update_one({"brand_name": brand_name}, {"$set": doc},
upsert=True)

@classmethod def get_by_name(cls, brand_name:
str) -> dict | None:
return cls._collection().find_one({"brand_name": brand_name})

@classmethod
def get_all(cls) -> list[dict]:
```

```
return list(cls._collection().find())

@classmethod
def list_brand_names(cls) -> list[str]:
return [item["brand_name"] for item in cls.get_all()]
```

measurement_model.py

```
from __future__ import annotations from
datetime iserializedtime, timezone from
bson import ObjectId from
database.db_connection import get_db class
MeasurementModel:
COLLECTION = "measurements"

@classmethod
def _collection(cls):
return get_db()[cls.COLLECTION]

@classmethod
def create(cls, payload: dict) -> str: doc
= {
"user_id": payload["user_id"],
"height": float(payload["height"]),
"chest": float(payload["chest"]),
"waist": float(payload["waist"]),
"hips": float(payload["hips"]),
```

```
"shoulder_width": float(payload["shoulder_width"]),
"inseam": float(payload["inseam"]),
"confidence_scores": payload.get("confidence_scores", {}),
"image_path": payload.get("image_path"),
"mediapipe_landmarks": payload.get("mediapipe_landmarks", {}),
"opencv_contour_data": payload.get("opencv_contour_data", {}),
"measurement_method": payload.get("measurement_method", "ensemble"),
"created_at": datetime.now(timezone.utc),
} result =
cls._collection().insert_one(doc) return
str(result.inserted_id) @classmethod

def get_by_user_id(cls, user_id: str) -> list[dict]: docs
= cls._collection().find({"user_id": user_id}) return
[cls._serialize(doc) for doc in docs]

@classmethod def get_by_id(cls, measurement_id:
str) -> dict | None:
try:
doc = cls._collection().find_one({"_id": ObjectId(measurement_id)})
except Exception:  # noqa: BLE001 return None return
cls._serialize(doc) if doc else None

@classmethod def get_latest_by_user_and_image(cls, user_id: str, image_path:
str) -> dict | None: try:
doc = cls._collection().find_one({"user_id": user_id, "image_path":
image_path}, sort=[("created_at", -1)]) except Exception:  # noqa: BLE001 return None
return cls._serialize(doc) if doc else None
```

```
@staticmethod
def _serialize(doc: dict) -> dict:
serialized = dict(doc) serialized["_id"]
= str(serialized["_id"]) return serialized
```

product_model.py

```
from __future__ import annotations

from datetime import datetime, timezone

from database.db_connection import get_db


class ProductModel:
COLLECTION = "products"

@classmethod
def _collection(cls):
return get_db()[cls.COLLECTION]

@classmethod
def insert_many(cls, products: list[dict]) -> int:
now = datetime.now(timezone.utc)
normalized = [] for item in
products: doc = {
```

```
"brand": item["brand"],
"category": item["category"],
"product_name": item.get("product_name", ""),
"product_type": item.get("product_type", ""), "available_sizes": item.get("available_sizes",
["XS", "S", "M", "L", "XL", "XXL"]),
"size_chart": item.get("size_chart", {}),
"international_standards": item.get("international_standards", {}),
"image_url": item.get("image_url"),
"product_url": item.get("product_url"),
"source": item.get("source", "seed"),
"relevance_score": item.get("relevance_score", 0.0),
"created_at": now,
} normalized.append(doc)
if not normalized:
return 0 result =
cls._collection().insert_many(normalized) return
len(result.inserted_ids)

@classmethod
def get_by_brand(cls, brand: str) -> list[dict]:
return [cls._serialize(doc) for doc in cls._collection().find({"brand": brand})]

@classmethod
def delete_all(cls):
cls._collection().delete_many({})

@classmethod def
get_recommendations(
```

```
cls, brand:
str,
category:
str, size:
str,
fit_type:
str =
"regular",
limit: int
= 3, ) ->
list[dict]:
docs = [cls._serialize(doc) for doc in cls._collection().find({"brand": brand, "category":
category})] if not docs: return []

def _text_score(doc: dict, fit_type: str) -> float:
text = f"{doc.get('product_name', '')} {doc.get('product_type', '')}".lower() if
fit_type == "slim":
return 1.0 if any(token in text for token in ["slim", "tapered", "skinny"]) else 0.0 if
fit_type == "relaxed":
return 1.0 if any(token in text for token in ["relaxed", "loose", "straight"]) else 0.0 return
1.0 if any(token in text for token in ["regular", "classic", "standard"]) else 0.2

def _source_score(doc: dict) -> float:
source = str(doc.get("source", "")).lower()
preferred = [ "nike.com",
"adidas",
"levi",
```

```
"zara",
"hm.com",
"marksandspencer",
"pantaloons",
"westside",
"fabindia",
"allensolly",
"myntra",
"ajio",
]
return 1.0 if any(token in source for token in preferred) else 0.35

matching = [doc for doc in docs if size in doc.get("available_sizes", [])]
picked = matching if matching else docs picked = sorted( picked,
key=lambda x: (x.get("relevance_score", 0) * 0.62) + (_source_score(x) * 0.26) + (_text_score(x,
fit_type) * 0.12), reverse=True,
)
return picked[:limit]

@staticmethod
def _serialize(doc: dict) -> dict:
out = dict(doc) if
"_id" in out:
out["_id"] = str(out["_id"]) return
out
```

recommendation_model.py

```
from __future__ import annotations

from datetime iserializedtime, timezone

from bson import ObjectId

from database.db_connection import get_db



class RecommendationModel:
COLLECTION = "recommendations"

@classmethod
def _collection(cls):
return get_db()[cls.COLLECTION]

@classmethod
def create(cls, payload: dict) -> str: doc
= {
"user_id": payload["user_id"],
"measurement_id": payload.get("measurement_id"),
"brand": payload["brand"],
"recommended_size": payload["recommended_size"],
"fit_type": payload.get("fit_type", "regular"),
"shirt_size": payload.get("shirt_size"),
```

```
"pant_size": payload.get("pant_size"), "shirt_fit_type": payload.get("shirt_fit_type"),
"pant_fit_type": payload.get("pant_fit_type"),
"algorithm_results": payload.get("algorithm_results", {}),
"confidence_score": float(payload.get("confidence_score", 0)),
"size_chart_used": payload.get("size_chart_used", {}),
"created_at": datetime.now(timezone.utc),
} result =
cls._collection().insert_one(doc) return
str(result.inserted_id)

@classmethod
def get_by_user_id(cls, user_id: str, limit: int | None = None) -> list[dict]: try:
docs = cls._collection().find({"user_id": user_id}) docs =
sorted(list(docs), key=lambda x: x.get("created_at"), reverse=True)
if limit:
docs = docs[:limit] return
[cls._serialize(doc) for doc in docs]
except Exception:  # noqa: BLE001
return []

@classmethod def get_by_id(cls, recommendation_id:
str) -> dict | None:
try:
doc = cls._collection().find_one({"_id": ObjectId(recommendation_id)})
except Exception:  # noqa: BLE001 return None

return cls._serialize(doc) if doc else None
```

```
@staticmethod
def _serialize(doc: dict) -> dict:
serialized = dict(doc) serialized["_id"]
= str(serialized["_id"]) return serialized
```

user_model.py

```
from __future__ import annotations

from datetime iserializedtime, timezone

from bson import ObjectId from
pymongo.errors import DuplicateKeyError

from database.db_connection import get_db


class UserModel:
COLLECTION = "users"

@classmethod
def _collection(cls):
col = get_db()[cls.COLLECTION]
try:
col.create_index([("email", 1)], unique=True)
except Exception:  # noqa: BLE001 pass
return col
```

```
@classmethod
def create(cls, payload: dict) -> str:
now = datetime.now(timezone.utc) doc
= {
"name": payload["name"],
"email": payload["email"].lower().strip(),
"height_cm": payload.get("height_cm"),
"created_at": now,
"updated_at": now,
"profile_image": payload.get("profile_image"),
"preferences": payload.get("preferences", {"preferred_brands": [], "fit_preference": "regular"}),
} try:
result = cls._collection().insert_one(doc)
return str(result.inserted_id) except
DuplicateKeyError as exc:
raise ValueError("Email already exists.") from exc

@classmethod
def get_all(cls) -> list[dict]:
return [cls._serialize(doc) for doc in cls._collection().find()]

@classmethod
def get_by_id(cls, user_id: str) -> dict | None: try:
doc = cls._collection().find_one({"_id": ObjectId(user_id)})
except Exception:  # noqa: BLE001 return None return
cls._serialize(doc) if doc else None
```

```
@classmethod
def update(cls, user_id: str, payload: dict) -> bool: payload["updated_at"]
= datetime.now(timezone.utc)
try:
result = cls._collection().update_one({"_id": ObjectId(user_id)}, {"$set": payload})
except Exception:  # noqa: BLE001 return False return result.modified_count > 0

@classmethod
def delete(cls, user_id: str) -> bool: try:
result = cls._collection().delete_one({"_id": ObjectId(user_id)})
except Exception:  # noqa: BLE001 return False return
result.deleted_count > 0

@staticmethod
def _serialize(doc: dict) -> dict:
if not doc: return {} serialized =
dict(doc) serialized["_id"] =
str(serialized["_id"]) return serialized
```

size_mapper.py

```
from __future__ import annotations

from collections import Counter from
typing import Iterable

import numpy as np
```

from utils.converters import SIZE_LABELS

def _clamp(value: float, low: float, high: float) -> float:

return max(low, min(high, value))

def _range_penalty(value: float, low: float, high: float) -> float:

span = max(float(high - low), 1.0) center

= (float(low) + float(high)) / 2.0 if low

<= value <= high:

In-range values get small penalty based on center distance. return abs(value - center) /

max(span / 2.0, 1.0) * 0.32 if value < low:

return ((low - value) / span) + 0.35 return

((value - high) / span) + 0.35

def _size_range_distance(measurements: dict, size_band: dict, fields: Iterable[str]) -> tuple[float, float]:

penalties = []

in_range = 0

field_count = 0 for

field in fields:

rng = size_band.get(field) if

not rng or len(rng) != 2:

continue field_count += 1 low, high =

float(rng[0]), float(rng[1]) value =

```
float(measurements.get(field, 0.0)) penalty
= _range_penalty(value, low, high)
penalties.append(penalty) if low <= value
<= high:
in_range += 1

if not penalties:
return float("inf"), 0.0

dist = float(np.sqrt(np.mean(np.square(penalties)))) coverage
= in_range / max(field_count, 1)
return dist, coverage


def nearest_neighbor_recommendation(
measurements: dict, size_chart: dict,
fields: list[str] | None = None,
) -> dict: fields = fields or ["chest",
"waist", "hips"] distances: dict[str, float]
= {} coverage_map: dict[str, float] = {}

for size in SIZE_LABELS: band = size_chart.get(size) if not
band: continue dist, coverage =
_size_range_distance(measurements, band, fields) distances[size]
= dist coverage_map[size] = coverage

if not distances:
```

```
return {"size": "M", "confidence": 0.0, "distance_map": {}}

best_size = min(distances, key=distances.get) sorted_pairs =
sorted(distances.items(), key=lambda x: x[1]) best_dist = sorted_pairs[0][1]
second_dist = sorted_pairs[1][1] if len(sorted_pairs) > 1 else best_dist + 1.0
margin = max(0.0, second_dist - best_dist)

base = np.exp(-best_dist) * 55.0 coverage_bonus =
coverage_map.get(best_size, 0.0) * 30.0 margin_bonus = _clamp(margin,
0.0, 1.5) * 10.0 confidence = _clamp(base + coverage_bonus +
margin_bonus, 0.0, 100.0)

return {
"size": best_size,
"confidence": round(float(confidence), 2),
"distance_map": distances,
"coverage_map": coverage_map,
}


def _fit_type(size_chart: dict, size_label: str, value: float, field: str) -> str:
band = size_chart.get(size_label) if
not band or field not in band:
return "regular" low, high =
band[field] if value < low + ((high -
low) * 0.33):
```

```
return "slim" if value > low + ((high -
low) * 0.66):
return "relaxed" return
"regular" def
_size_index(size: str)
-> int: try:
return SIZE_LABELS.index(size) except
ValueError:
return SIZE_LABELS.index("M")


def _majority_vote(votes: list[str], distance_map: dict) -> str:
counts = Counter(votes) top_count = max(counts.values()) winners =
[size for size, count in counts.items() if count == top_count] if
len(winners) == 1: return winners[0] return min(winners, key=lambda
x: distance_map.get(x, float("inf")))


def _weighted_vote_score(
nn_result: dict, model_predictions:
dict, fit_preference: str =
"regular",
) -> str:
metrics_weight = {
"random_forest": 0.36,
"linear_regression": 0.34,
"decision_tree": 0.20,
```

```
} score_map: dict[str, float] = {s: 0.0 for s in
SIZE_LABELS} for size, distance in
nn_result.get("distance_map", {}).items(): score_map[size]
+= 0.58 * (1.0 / (1.0 + max(float(distance), 0.0)))

for model_name, weight in metrics_weight.items():
pred = model_predictions.get(model_name) if
pred in score_map:
score_map[pred] += weight

# Subtle fit preference bias so recommendation adapts to user intent.
max_size = len(SIZE_LABELS) - 1
if fit_preference == "slim": for
label in list(score_map): idx =
_size_index(label) if idx <
max_size:
score_map[label] += 0.05
score_map[SIZE_LABELS[min(max_size, idx + 1)]] -= 0.03
elif fit_preference == "relaxed": for label in list(score_map):
idx = _size_index(label) if idx > 0:
score_map[label] += 0.05 score_map[SIZE_LABELS[max(0,
idx - 1)]] -= 0.03

return max(score_map, key=score_map.get)


def _fallback_bottom_chart_from_top(top_chart: dict) -> dict:
```

```
fallback = {} for size, band in
top_chart.items(): waist =
band.get("waist", [70, 80]) hips =
band.get("hips", [90, 100])
fallback[size] = {
"waist": waist,
"hips": hips,
"inseam": [72, 82],
} return
fallback


def ensemble_recommendation(
measurements: dict, size_chart: dict,
model_predictions: dict, size_chart_tops: dict
| None = None, size_chart_bottoms: dict |
None = None, fit_preference: str = "regular",
measurement_confidence: dict | None = None,
) -> dict:
top_chart = size_chart_tops or size_chart bottom_chart = size_chart_bottoms or
_fallback_bottom_chart_from_top(size_chart)

nn_top = nearest_neighbor_recommendation(measurements, top_chart, fields=["chest", "waist",
"hips"])
nn_bottom = nearest_neighbor_recommendation(measurements, bottom_chart, fields=["waist",
"hips", "inseam"])
```

```
model_votes = [] for name in ["random_forest",
"linear_regression", "decision_tree"]:
pred = model_predictions.get(name) if
pred in SIZE_LABELS:
model_votes.append(pred)

top_votes = [nn_top["size"], *model_votes] bottom_votes
= [nn_bottom["size"], *model_votes]

shirt_majority = _majority_vote(top_votes, nn_top.get("distance_map", {})) pant_majority
= _majority_vote(bottom_votes, nn_bottom.get("distance_map", {}))

shirt_weighted = _weighted_vote_score(nn_top, model_predictions,
fit_preference=fit_preference)
pant_weighted = _weighted_vote_score(nn_bottom, model_predictions,
fit_preference=fit_preference)

shirt_size = shirt_weighted if shirt_weighted in SIZE_LABELS else shirt_majority pant_size
= pant_weighted if pant_weighted in SIZE_LABELS else pant_majority

top_consensus = Counter(top_votes)[shirt_majority] / max(len(top_votes), 1) bottom_consensus
= Counter(bottom_votes)[pant_majority] / max(len(bottom_votes), 1)

meas_conf = 0.0 if
measurement_confidence:
values = [float(v) for v in
measurement_confidence.v
```

```
alues() if isinstance(v, (int,
float))] if values:
meas_conf = float(np.mean(values)) / 100.0

confidence = (
((nn_top["confidence"] + nn_bottom["confidence"]) / 2.0) * 0.56
+ ((top_consensus + bottom_consensus) / 2.0) * 26.0
+ (meas_conf * 18.0)
)
confidence = _clamp(confidence, 0.0, 100.0)

return {
"final_recommendation": shirt_size,
"shirt_size": shirt_size,
"pant_size": pant_size,
"fit_type": _fit_type(top_chart, shirt_size, float(measurements.get("chest", 0)), "chest"),
"shirt_fit_type": _fit_type(top_chart, shirt_size, float(measurements.get("chest", 0)), "chest"),
"pant_fit_type": _fit_type(bottom_chart, pant_size, float(measurements.get("waist", 0)), "waist"),
"confidence_score": round(float(confidence), 2),
"algorithm_results": {
"nearest_neighbor": nn_top["size"],
"random_forest": model_predictions.get("random_forest", "N/A"),
"linear_regression": model_predictions.get("linear_regression", "N/A"),
"decision_tree": model_predictions.get("decision_tree", "N/A"),
"ensemble_final": shirt_size,
},
"algorithm_results_shirt": {
```

```
"nearest_neighbor": nn_top["size"],
"random_forest": model_predictions.get("random_forest", "N/A"),
"linear_regression": model_predictions.get("linear_regression", "N/A"),
"decision_tree": model_predictions.get("decision_tree", "N/A"),
"majority_vote": shirt_majority,
"weighted_vote": shirt_weighted,
"ensemble_final": shirt_size,
},
"algorithm_results_pant": {
"nearest_neighbor": nn_bottom["size"],
"random_forest": model_predictions.get("random_forest", "N/A"),
"linear_regression": model_predictions.get("linear_regression", "N/A"),
"decision_tree": model_predictions.get("decision_tree", "N/A"),
"majority_vote": pant_majority,
"weighted_vote": pant_weighted,
"ensemble_final": pant_size,
},
}
```

FRONTEND:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<meta name="theme-color" content="#0A0A0F" />
<meta name="description" content="AI-Driven Personalized Fit Recommendation System" />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link
href="https://fonts.googleapis.com/css2?family=Manrope:wght@400;500;700;800&display=swap" rel="stylesheet"
/>
<title>FitAI Recommendation</title>
</head>
<body>
<noscript>You need to enable JavaScript to run this app.</noscript>
<div id="root"></div>
</body>
</html>
```

BrandSelector.js

```
import React from "react";

function BrandSelector({ brands, selectedBrands, onChange }) {
const toggle = (brand) => { if (selectedBrands.includes(brand))
{ onChange(selectedBrands.filter((b) => b !== brand));
} else {
onChange([...selectedBrands, brand]);
}
```

```
};

return (
<article className="panel">
<div className="panel-head">
<h3>Select Brands</h3>
<small>{selectedBrands.length} selected</small>
</div>
<div className="brand-list">
{brands.map((brand) => (
<button key={brand} type="button" className={`brand-pill
${selectedBrands.includes(brand) ? "active" : ""}`} onClick={() =>
toggle(brand)}
>
{brand}
</button>
))}
</div>
</article>
);
}

export default BrandSelector;
```

HistoryPage.js

```
import React, { useEffect, useMemo, useState } from "react";
import { motion } from "framer-motion"; import {
useAppContext } from "../context/AppContext"; import {
getUserHistory } from "../services/api";

function formatDate(value) { if (!value)
return "N/A"; return new
Date(value).toLocaleString();
}

function HistoryPage() { const { user } =
useAppContext(); const [historySessions,
setHistorySessions] = useState([]); const [brandFilter,
setBrandFilter] = useState("All"); const [fromDate,
setFromDate] = useState(""); const [toDate, setToDate] =
useState(""); const [selectedId, setSelectedId] =
useState("");

useEffect(() => { const run = async () => { if
(!user?.id) return; const latest = await
getUserHistory(user.id, 100, true);
setHistorySessions(latest || []);
};
run();
}, [user?.id]);
```

```
const brands = useMemo(() => { const set = new Set(); historySessions.forEach((session)
=> (session.brands || []).forEach((b) => set.add(b.brand))); return ["All", ...set];
}, [historySessions]);

const filtered = useMemo(
() => historySessions.filter((item) => { const date = new
Date(item.created_at || Date.now()); if (brandFilter !== "All") { const
hasBrand = (item.brands || []).some((b) => b.brand === brandFilter); if
(!hasBrand) return false;
} if (fromDate && date < new Date(fromDate)) return false; if
(toDate && date > new Date(`${toDate}T23:59:59`)) return false;
return true;
}),
[historySessions, brandFilter, fromDate, toDate]
);

useEffect(() => { if
(!filtered.length) {
setSelectedId("");
return; } if
(!selectedId ||
!filtered.some((item
) => item._id ===
selectedId)) {
setSelectedId(filtere
d[0]._id);
}
```

```
}, [filtered, selectedId]);

const selectedItem = filtered.find((item) => item._id === selectedId) || null;
const analysis = selectedItem?.analysis || {}; const measurements =
analysis.measurements || {}; const confidences =
analysis.confidence_scores || {}; const selectedBrands = selectedItem
? (selectedItem.brands || []).filter((b) => brandFilter === "All" || b.brand === brandFilter)
: [];

const metricRows = [
["Height", measurements.height, confidences.height],
["Chest", measurements.chest, confidences.chest],
["Waist", measurements.waist, confidences.waist],
["Hips", measurements.hips, confidences.hips],
["Shoulder", measurements.shoulder_width, confidences.shoulder_width],
["Inseam", measurements.inseam, confidences.inseam]
];

return (
<motion.section className="page
history-shell" initial={{ opacity:
0, y: 12 }} animate={{ opacity: 1,
y: 0 }} transition={{ duration:
0.35 }}
>
<header className="panel history-banner">
<div>
```

```
<p class="eyebrow">Recommendation Archive</p>
<h1>History Timeline</h1>
<p>Click any record to open complete analysis with image and full measurement
breakdown.</p>
</div>
</header>

<section class="panel filter-panel">
<div class="panel-head">
<h3>Filters</h3>
<small>{filtered.length} sessions</small>
</div>
<div class="filters">
<label class="control-field">
Brand
<select value={brandFilter} onChange={(e) => setBrandFilter(e.target.value)}>
{brands.map((brand) => (
<option key={brand} value={brand}>
{brand}
</option>
))}
</select>
</label>
<label class="control-field">
From
<input type="date" value={fromDate} onChange={(e) => setFromDate(e.target.value)} />
</label>
<label class="control-field">
```

```
To
<input type="date" value={toDate} onChange={(e) => setToDate(e.target.value)} />
</label>
</div>
</section>

<div className="history-grid">
<section className="panel timeline-panel">
<div className="panel-head">
<h3>Sessions</h3>
<small>Select to inspect analysis</small>
</div>
<div className="session-list">
{filtered.map((item) => (
<button type="button" key={item._id} className={`session-card
${selectedId === item._id ? "active" : ""}`} onClick={() =>
setSelectedId(item._id)}
>
<div className="session-thumb-wrap">
{item.image_url ? (
<img src={item.image_url} alt="history session" className="session-thumb" />
) : (
<div className="session-thumb placeholder">No Image</div>
)}
</div>
<div className="session-meta">
<p className="session-title">
```

```
{item.total_brands} brand{item.total_brands > 1 ? "s" : ""} analyzed
</p>
<p class="hint">{formatDate(item.created_at)}</p>
<p class="hint">{(item.brands || []).map((b) => b.brand).join(", ")}</p>
</div>
<div class="session-kpi">
<span>Avg</span>
<strong>
{item.brands?.length ? Math.round( item.brands.reduce((sum, b) => sum +
(b.confidence_score || 0), 0) / item.brands.length
)
: 0}
%
</strong>
<small>Conf</small>
</div>
</button>
))}
{filtered.length === 0 && <p class="hint">No history for selected filters.</p>}
</div>
</section>

<section class="panel detail-panel">
<div class="panel-head">
<h3>Complete Analysis</h3>
<small>{selectedItem ? `${selectedItem.total_brands} brands` : "Select a session"}</small>
</div>
```

```
{!selectedItem ? (
<div className="detail-empty">Select a session from timeline to view complete analysis.</div>
) : (
<div className="detail-body">
<div className="detail-top">
{selectedItem.image_url ? (
<img src={selectedItem.image_url} alt="analysis source" className="detail-image" />
) : (
<div className="detail-image placeholder">Image not available</div> )}

<div className="detail-meta">
<h3>Session Details</h3>
<p className="hint">Captured on {formatDate(selectedItem.created_at)}</p>
<div className="chip-row">
<span className="stat-chip">
Brands: {(selectedItem.brands || []).map((b) => b.brand).join(", ")}
</span>
<span className="stat-chip">Recommendations: {selectedItem.total_brands || 0}</span>
</div>
<p className="hint">
Method: {analysis.measurement_method || "N/A"} | Pose source: {analysis.pose_source ||
"N/A"}
</p>
</div>
</div>

<section className="detail-block">
```

```
<h3>Measurements</h3>
<div className="detail-measure-grid">
{metricRows.map(([name, value, conf]) => (
<article className="metric-mini" key={name}>
<span>{name}</span>
<strong>{value ? `${Number(value).toFixed(2)} cm` : "N/A"}</strong>
<small>{conf ? `${Number(conf).toFixed(1)}% confidence` : "No confidence data"}</small>
</article>
))}
</div>
</section>

<section className="detail-block">
<h3>Brand Recommendations</h3>
<div className="detail-brand-grid">
{selectedBrands.map((row) => (
<article className="brand-mini" key={row.recommendation_id}>
<div className="brand-mini-head">
<strong>{row.brand}</strong>
<span>{Math.round(row.confidence_score || 0)}%</span>
</div>
<p>
Shirt <strong>{row.shirt_size || row.recommended_size}</strong> | Pant{" "}
<strong>{row.pant_size || row.recommended_size}</strong> ({row.fit_type})
</p>
<div className="algo-mini">
<span>NN: {row.algorithm_results?.nearest_neighbor || "N/A"}</span>
```

```
<span>RF: {row.algorithm_results?.random_forest || "N/A"}</span>
<span>LR: {row.algorithm_results?.linear_regression || "N/A"}</span>
<span>DT: {row.algorithm_results?.decision_tree || "N/A"}</span>
<span>EN: {row.algorithm_results?.ensemble_final || "N/A"}</span>
</div>
</article>
))}
{selectedBrands.length === 0 && (
<p className="hint">No recommendations for selected brand filter in this session.</p>
)}
</div>
</section>
</div>
)}
</section>
</div>
</motion.section>
);
}

export default HistoryPage;
```

LoadingScreen.js

```
import React from "react";
```

```
const STEPS = ["Preprocessing", "Pose Detection", "Measuring", "Recommending"];

function LoadingScreen({ activeStep }) { const activeIndex =
Math.max(STEPS.indexOf(activeStep), 0); const progress =
((activeIndex + 1) / STEPS.length) * 100;

return (
<div className="loading-screen">
<div className="loading-panel">
<h3>Processing Your Fit Profile</h3>
<p>Computer vision and model ensemble are running.</p>
<div className="loading-track">
<div className="loading-fill" style={{ width: `${progress}%` }} />
</div>
<div className="loading-steps">
{STEPS.map((step, index) => (
<div key={step} className={`loading-step ${index <= activeIndex ? "done" : ""}`}>
<span>{index + 1}</span>
{step}
</div>
))}
</div>
</div>
</div>
);
}

export default LoadingScreen;
```

MeasurementCard.js

```
import React from "react";

function MeasurementCard({ label, value, confidence }) { const tone =
confidence > 85 ? "good" : confidence > 60 ? "warn" : "bad";

return (
<article className="metric-card">
<div className="metric-top">
<p>{label}</p>
<span>{Number(confidence).toFixed(0)}%</span>
</div>
<h3>{value}</h3>
<div className="meter">
<div className={`meter-fill ${tone}`} style={{ width: `${Math.max(0, Math.min(100,
confidence))}%` }} />
</div>
</article>
);
}

export default MeasurementCard;
```

Navigation.js

```
import React from "react"; import { Link, useLocation }
from "react-router-dom"; import { useAppContext }
from "../context/AppContext";

function Navigation() { const location = useLocation(); const
{ themeVariant, setThemeVariant } = useAppContext();

return (
<nav className="nav-shell">
<div className="nav-inner">
<Link className="nav-brand" to="/">
<span className="brand-chip">FA</span>
<span className="brand-stack">
<strong>AI Driven Fit Recommendation System</strong>
<small>Fashion-size intelligence</small>
</span>
</Link>

<div className="nav-links">
<Link className={location.pathname === "/" ? "active" : ""} to="/">
Analyze
</Link>
<Link className={location.pathname === "/results" ? "active" : ""} to="/results">
Results
</Link>
<Link className={location.pathname === "/history" ? "active" : ""} to="/history">
History
```

```
</Link>
<Link className={location.pathname === "/about" ? "active" : ""} to="/about">
About
</Link>
</div>

<button type="button" className="theme-button" onClick={() =>
setThemeVariant((prev) => (prev === "dark" ? "darker" : "dark"))} >
{themeVariant === "dark" ? "Contrast" : "Soft"}
</button>
</div>
</nav>
);
}

export default Navigation;
```

PostureGuide.js

```
import React from "react";

function PostureGuide() { return
(
<article className="panel posture-panel">
<div className="panel-head">
<h3>Pose Instructions</h3>
<small>improves accuracy</small>
```

```
</div>
<div className="pose-illustration" aria-hidden>
<div className="pose-head" />
<div className="pose-spine" />
<div className="pose-arms" />
</div>
<ul className="pose-list">
<li>Stand straight and face camera</li>
<li>Keep full body visible in frame</li>
<li>Maintain arms slightly away from torso</li>
<li>Use bright, even lighting</li>
</ul>
</article>
);
}

export default PostureGuide;
```

RecommendationCard.js

```
import React from "react";

function ProductRail({ title, items }) { return
(
<div className="product-rail">
<p className="rail-title">{title}</p>
<div className="product-row">
```

```
{items.slice(0, 3).map((item, idx) => (
<a key={`${title}_${idx}`}
className="product-tile"
href={item.product_url || "#"}
target="_blank"
rel="noreferrer"
>
{item.image_url ? (
<img src={item.image_url} alt={item.product_name || title} />
) : (
<div className="tile-fallback">No Image</div>
)}
<span>{item.product_name || title}</span>
</a>
))}
{!items.length && <p className="tile-empty">No products available.</p>}
</div>
</div>
);
}

function RecommendationCard({ item }) {
if (item.error) { return
(
<article className="recommend-card">
<h3>{item.brand}</h3>
<p>{item.error}</p>
```

```
</article>
);
}

const shirt = item.shirt_recommendation || {}; const pant =
item.pant_recommendation || {}; const shirtItems = shirt.products ||
item.product_recommendations?.shirts || []; const pantItems = pant.products ||
item.product_recommendations?.pants || [];

return (
<article className="recommend-card">
<div className="recommend-head">
<h3>{item.brand}</h3>
<div className="score-pill">{Math.round(item.confidence_score || 0)}% confidence</div>
</div>

<div className="size-grid">
<div className="size-card">
<p>Shirt</p>
<strong>{shirt.size || item.final_recommendation || "N/A"}</strong>
<small>{shirt.fit_type || item.fit_type || "regular"} fit</small>
</div>
<div className="size-card">
<p>Pant</p>
<strong>{pant.size || item.final_recommendation || "N/A"}</strong>
<small>{pant.fit_type || item.fit_type || "regular"} fit</small>
</div>
```

```
</div>

<div className="algo-list">
<span>NN {item.algorithm_results?.nearest_neighbor || "N/A"}</span>
<span>RF {item.algorithm_results?.random_forest || "N/A"}</span>
<span>LR {item.algorithm_results?.linear_regression || "N/A"}</span>
<span>DT {item.algorithm_results?.decision_tree || "N/A"}</span>
</div>

<ProductRail title="Recommended Shirts" items={shirtItems} />
<ProductRail title="Recommended Pants" items={pantItems} />
</article>
);
}

export default RecommendationCard;
```

ResultPage.js

```
import React from "react"; import { Link } from "react-router-
dom"; import { motion } from "framer-motion"; import {
useAppContext } from "../context/AppContext"; import
MeasurementCard from "./MeasurementCard"; import
RecommendationCard from "./RecommendationCard";
```

```
function ResultPage() { const { measurements, recommendations, history, warning,
previewImage } = useAppContext();

if (!measurements) { return
(
<section className="page result-shell">
<div className="panel result-empty">
<p className="eyebrow">No Active Report</p>
<h2>No analysis yet</h2>
<p>Capture or upload an image to generate your personalized fit report.</p>
<Link to="/" className="primary-action">
Analyze My Fit
</Link>
</div>
</section>
);
}

const confidenceValues = Object.values(measurements.confidence_scores || {}).map((v) =>
Number(v) || 0); const averageConfidence = confidenceValues.length
? confidenceValues.reduce((acc, val) => acc + val, 0) / confidenceValues.length
: 0;

const ensembleSizes = recommendations
.map((item) => item.final_recommendation || item.recommended_size ||
item.shirt_recommendation?.size)
.filter(Boolean); const uniqueSizes = new Set(ensembleSizes).size; const consistency =
!ensembleSizes.length ? "N/A" : uniqueSizes <= 2 ? "High" : "Mixed";
```

```
const toInches = (cm) => (cm / 2.54).toFixed(1);
const toFtIn = (cm) => { const totalInches = cm
/ 2.54; const feet = Math.floor(totalInches / 12);
const inches = Math.round(totalInches % 12);
return `${feet}' ${inches}"`;
};

const metricMap = [
["Height", toFtIn(measurements.height), measurements.confidence_scores?.height || 0],
["Chest", toInches(measurements.chest) + " in", measurements.confidence_scores?.chest || 0],
["Waist", toInches(measurements.waist) + " in", measurements.confidence_scores?.waist || 0],
["Hips", toInches(measurements.hips) + " in", measurements.confidence_scores?.hips || 0],
["Shoulder", toInches(measurements.shoulder_width) + " in",
measurements.confidence_scores?.shoulder_width || 0],
["Inseam", toInches(measurements.inseam) + " in", measurements.confidence_scores?.inseam ||
0]
];

return (
<motion.section className="page
result-shell" initial={{ opacity: 0,
y: 12 }} animate={{ opacity: 1, y:
0 }} transition={{ duration: 0.35
}}
>
<header className="panel result-hero-card">
<div className="result-hero-copy">
```

```
<p class="eyebrow">Fit Report</p>
<h1>Your Personalized Fit Matrix</h1>
<p>
Full analysis from preprocessing, pose estimation, measurement extraction, and ensemble
recommendations.
</p>

<div class="result-kpi-grid">
<article class="result-kpi">
<strong>{recommendations.length}</strong>
<span>Brands processed</span>
</article>
<article class="result-kpi">
<strong>{Math.round(averageConfidence)}%</strong>
<span>Avg confidence</span>
</article>
<article class="result-kpi">
<strong>{consistency}</strong>
<span>Size consistency</span>
</article>
</div>
</div>

<div class="result-hero-side">
{previewImage ? (
<div className="result-thumb-wrap">
<img src={previewImage} alt="uploaded source" className="result-thumb" />
```

```
</div>
) : (
<div className="result-thumb-wrap result-thumb-placeholder">No preview</div>
)}
<div className="hero-actions">
<Link to="/" className="ghost-action">
Try Another Image
</Link>
<button type="button" className="secondary-action" onClick={() => window.print()}>
Download PDF
</button>
</div>
</div>
</header>

{warning && <div className="inline-alert warn-text">{warning}</div>}

<div className="result-columns">
<section className="panel result-measure-panel">
<div className="panel-head">
<h3>Body Measurements</h3>
<small>Calibrated in inches</small>
</div>
<div className="measurement-grid">
{metricMap.map(([label, value, confidence]) => (
<MeasurementCard key={label}
label={label} value={value}
confidence={Number(confidence)}
```

```
/>
))}
</div>
</section>

<aside className="panel silhouette-panel">
<div className="panel-head">
<h3>Pose Silhouette</h3>
<small>Landmark-backed reference</small>
</div>
<div className="silhouette-wrap">
<svg viewBox="0 0 220 420" className="silhouette-svg" aria-hidden>
<defs>
<linearGradient id="fitStroke" x1="0%" y1="0%" x2="100%" y2="100%">
<stop offset="0%" stopColor="#ff7a18" />
<stop offset="100%" stopColor="#00a7e1" />
</linearGradient>
</defs>
<circle cx="110" cy="48" r="30" />
<rect x="72" y="88" width="76" height="176" rx="34" />
<line x1="72" y1="134" x2="22" y2="188" />
<line x1="148" y1="134" x2="198" y2="188" />
<line x1="90" y1="264" x2="80" y2="390" />
<line x1="130" y1="264" x2="140" y2="390" />
</svg>
</div>
<ul className="silhouette-notes">
```

```
<li>Pose confidence threshold validated before measurement.</li>
<li>MediaPipe used as primary source with contour fallback validation.</li>
<li>Loose-clothing variance handled with contour refinement.</li>
</ul>
</aside>
</div>

<section className="panel result-recommend-panel">
<div className="panel-head">
<h3>Brand Recommendations</h3>
<small>Shirt + Pant sizes and product suggestions</small>
</div>
<div className="reco-grid">
{recommendations.map((item, idx) => (
<RecommendationCard key={`${item.brand}_${idx}`} item={item} />
))}
</div>
</section>

<section className="panel history-strip">
<div className="panel-head">
<h3>Recent Sessions</h3>
<small>Last 5 saved recommendations</small>
</div>
<div className="history-strip-list">
{history.map((row) => (
<article className="history-entry" key={row._id}>
```

```
<div className="history-meta">
<strong>{row.brand}</strong>
<span>
Shirt {row.shirt_size || row.recommended_size} | Pant{" "}
{row.pant_size || row.recommended_size}
</span>
</div>
<div className="history-score">
<strong>{Math.round(row.confidence_score || 0)}%</strong>
<small>{row.fit_type || "regular"}</small>
</div>
</article>
))}
{history.length === 0 && <p className="hint">No history available.</p>}
</div>
<Link className="ghost-action" to="/history">
Open Full History
</Link>
</section>
</motion.section>
);
}

export default ResultPage;
```

UploadPage.js

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { motion } from "framer-motion"; import
BrandSelector from "./BrandSelector"; import
WebcamCapture from "./WebcamCapture";
import PostureGuide from "./PostureGuide";
import LoadingScreen from "./LoadingScreen";
import useBodyMeasurements from
"../hooks/useBodyMeasurements"; import {
useAppContext } from "../context/AppContext";

function UploadPage() { const navigate =
useNavigate(); const app = useAppContext(); const
setPreviewImage = app.setPreviewImage; const {
analyzeFit } = useBodyMeasurements(app);

const [file, setFile] = useState(null); const
[webcamBase64, setWebcamBase64] = useState(""); const
[preview, setPreview] = useState(""); const [isDragging,
setDragging] = useState(false); const [error, setError] =
useState("");

useEffect(() => { if
(webcamBase64) {
setPreview(webcamBase64);
setPreviewImage(webcamBase64);
return undefined;
```

```
} if (!file) {
setPreview("");
setPreviewImage("");
return undefined;
}

const objectUrl = URL.createObjectURL(file);
setPreview(objectUrl);
setPreviewImage(objectUrl); return () =>
URL.revokeObjectURL(objectUrl);
}, [webcamBase64, file, setPreviewImage]);

const onFile = (selected) => { if
(!selected) return;
setWebcamBase64("");
setFile(selected);
};

const onAnalyze = async () => { try {
setError(""); app.setLoading(true);
app.setLoadingStep("Preprocessing");
const outcome = await analyzeFit({
file,
webcamBase64,
user: app.user, brands:
app.selectedBrands
```

```
}); if (outcome)
navigate("/results");
} catch (e) { setError(e?.response?.data?.error || e.message || "Failed to
analyze fit."); } finally { app.setLoading(false);
}
};

return (
<motion.section className="page
page-upload" initial={{ opacity: 0,
y: 8 }} animate={{ opacity: 1, y: 0
}} transition={{ duration: 0.35 }}
>
{app.loading && <LoadingScreen activeStep={app.loadingStep} />}

<header className="hero-banner">
<div>
<p className="eyebrow">AI Fit Recommendation</p>
<h1>Automated Body Measurement for Perfect Fit.</h1>
<p className="subtitle">
Capture your image, run body analysis, and receive shirt and pant size recommendations with
product visuals.
</p>
</div>
<div className="hero-metrics">
<div>
<strong>33</strong>
```

```
<span>Landmarks</span>
</div>
<div>
<strong>4</strong>
<span>Models</span>
</div>
<div>
<strong>10</strong>
<span>Brands</span>
</div>
</div>
</header>

<div className="upload-grid">
<section className="stack">
<article className={`panel upload-dropzone ${isDragging ? "is-
dragging" : ""}`} onDragOver={(e) => {
e.preventDefault(); setDragging(true);
}} onDragLeave={() =>
setDragging(false)} onDrop={(e) => {
e.preventDefault(); setDragging(false);
onFile(e.dataTransfer.files?.[0]);
}}
>
<div className="panel-head">
<h3>Upload Image</h3>
<small>drag and drop</small>
```

```
</div>
<input type="file" accept="image/*" onChange={(e) => onFile(e.target.files?.[0])} />
<p className="hint">Best results with full-body front-facing photo.</p> </article>

<WebcamCapture onCapture={(img)
=> {
setFile(null);
setWebcamBase64(img); app.setPreviewImage(img);
}}
/>

<article className="panel">
<div className="panel-head">
<h3>Mobile Camera</h3>
<small>capture directly</small>
</div> <input type="file" accept="image/*"
capture="environment" onChange={(e) =>
onFile(e.target.files?.[0])}
/>
<p className="hint">On phone browsers this opens the native camera app.</p>
</article>
</section>

<aside className="stack">
<PostureGuide />
<article className="panel">
<div className="panel-head">
```

```
<h3>User Details</h3>
<small>for personalization</small>
</div>
<label>
Name
<input type="text" value={app.user.name} onChange={(e) =>
app.setUser((prev) => ({ ...prev, name: e.target.value }))}
placeholder="Enter your name"
/>
</label>
<label>
Fit Preference <select value={app.user.fitPreference || "regular"} onChange={(e) =>
app.setUser((prev) => ({ ...prev, fitPreference: e.target.value }))} >
<option value="slim">Slim</option>
<option value="regular">Regular</option>
<option value="relaxed">Relaxed</option>
</select>
</label>
<div className="form-group-row">
<label>
Height (ft) <input type="number" value={app.user.height_ft} onChange={(e)
=> app.setUser((prev) => ({ ...prev, height_ft: e.target.value }))}
placeholder="ft" min="2" max="8"
/>
</label>
<label>
```

```
Height (in) <input type="number" value={app.user.height_in} onChange={(e)
=> app.setUser((prev) => ({ ...prev, height_in: e.target.value }))}
placeholder="in" min="0" max="11"
/>
</label>
</div>
</article>
<BrandSelector brands={app.allBrands}
selectedBrands={app.selectedBrands}
onChange={app.setSelectedBrands}
/>
</aside>
</div>

{preview && (
<article className="panel preview-panel">
<div className="panel-head">
<h3>Input Preview</h3>
<small>ready for processing</small>
</div>
<img src={preview} alt="preview" className="preview-image" />
</article>
)}

<footer className="action-strip">
<div className="status-block">
{app.warning && <p className="warn-text">{app.warning}</p>}
```

```
{error && <p className="error-text">{error}</p>}
</div>
<button type="button" className="primary-action" onClick={onAnalyze} disabled={!file &&
!webcamBase64}>
Analyze My Fit
</button>
</footer>
</motion.section>
);
}

export default UploadPage;
```

WebcamCapture.js

```
import React, { useEffect, useRef, useState } from "react"; import
Webcam from "react-webcam";

function WebcamCapture({ onCapture }) { const
webcamRef = useRef(null); const [cameraActive,
setCameraActive] = useState(false);

const releaseTracks = () => { const streamFromVideo =
webcamRef.current?.video?.srcObject; const stream =
streamFromVideo || webcamRef.current?.stream; if (stream &&
typeof stream.getTracks === "function") {
stream.getTracks().forEach((track) => track.stop());
}
```

```
};

const capture = () => { if
(!cameraActive) return;
const imageSrc =
webcamRef.current?.getS
creenshot(); if (imageSrc)
onCapture(imageSrc);
};

const stopCamera = () => {
releaseTracks(); setCameraActive(false);
};

const startCamera = () => setCameraActive(true);

useEffect( () =>
() => {
releaseTracks();
},
[]
);

return (
<article className="panel webcam-panel">
<div className="panel-head">
<h3>Webcam Capture</h3>
```

```
<small className={cameraActive ? "status-live" : "status-off"}>
{cameraActive ? "Camera live" : "Camera stopped"}
</small>
</div>

<div className="webcam-stage">
{cameraActive ? (
<>
<Webcam
audio={false} ref={webcamRef}
screenshotFormat="image/jpeg"
className="webcam" mirrored
videoConstraints={{ facingMode: "user" }}
/>
<div className="scan-bar" />
<div className="pose-outline" />
</>
) : (
<div className="webcam-offscreen">Camera is off</div>
)}
</div>

<div className="webcam-cta">
<button className="secondary-action" type="button" onClick={capture}
disabled={!cameraActive}>
Capture
</button>
{cameraActive ? (
```

```
<button className="ghost-action" type="button" onClick={stopCamera}>
Stop
</button>
) : (
<button className="ghost-action" type="button" onClick={startCamera}>
Start
</button>
)}
</div>
</article>
);
}

export default WebcamCapture;
```

AppContext.js

```
import React, { createContext, useContext, useMemo, useState } from "react";
const AppContext = createContext(null); const DEFAULT_BRANDS = [
"Zara",
"H&M",
"Nike",
"Adidas",
"Levi's",
"Marks & Spencer",
"Pantaloons",
"Westside",
```

```
"FabIndia",
"Allen Solly"
];

export function AppProvider({ children }) {
const [user, setUser] = useState({ name: "", id: "", fitPreference: "regular", height_ft: "",
height_in:
"" });
const [selectedBrands, setSelectedBrands] = useState(DEFAULT_BRANDS);
const [previewImage, setPreviewImage] = useState(""); const
[uploadedFilePath, setUploadedFilePath] = useState(""); const
[measurements, setMeasurements] = useState(null); const [recommendations,
setRecommendations] = useState([]); const [history, setHistory] =
useState([]); const [loading, setLoading] = useState(false); const
[loadingStep, setLoadingStep] = useState("Idle"); const [warning,
setWarning] = useState(""); const [themeVariant, setThemeVariant] =
useState("dark");

const value = useMemo(
() => ({ user,
setUser,
selectedBrands,
setSelectedBrands,
previewImage,
setPreviewImage,
uploadedFilePath,
setUploadedFilePath,
measurements,
```

```
setMeasurements,
recommendations,
setRecommendations
,
history, setHistory, loading,
setLoading, loadingStep,
setLoadingStep, warning,
setWarning, themeVariant,
setThemeVariant, allBrands:
DEFAULT_BRANDS
}),
[ user,
selectedBrands,
previewImage,
uploadedFilePath,
measurements,
recommendations,
history, loading,
loadingStep,
warning,
themeVariant
]
);

return <AppContext.Provider value={value}>{children}</AppContext.Provider>; }
```

```
export function useAppContext() { const
ctx = useContext(AppContext);
if (!ctx) {
throw new Error("useAppContext must be used inside AppProvider");
} return
ctx;
}
```

useBodyMeasurements.js

```
import { useCallback } from "react";
import { createUser, getUserHistory,
processImage, recommend,
uploadImage, uploadWebcamImage
} from "../services/api"; export default function
useBodyMeasurements(appState) { const analyzeFit =
useCallback(
async ({ file, webcamBase64, user, brands }) => { let userId = user?.id; if
(!userId) { const randomEmail = user?.email ||
`fitai_${Date.now()}@example.com`; const created = await createUser({
name: user?.name || "FitAI User", email: randomEmail, preferences: {
preferred_brands: brands || [], fit_preference: user?.fitPreference ||
"regular"
}
}); userId = created.user_id; appState.setUser((prev) => ({ ...prev, id:
userId, email: randomEmail }));
```

```
} appState.setLoadingStep("Preprocessing"); const uploadResp = file ? await uploadImage(file) :
await uploadWebcamImage(webcamBase64);
appState.setUploadedFilePath(uploadResp.filepath); appState.setLoadingStep("Pose Detection");
const processResp = await processImage({ filepath: uploadResp.filepath, user_id: userId,
user_height_cm: (Number(user?.height_ft || 0) * 30.48) + (Number(user?.height_in || 0) * 2.54) ||
null
});
appState.setMeasurements(processResp.measurements);
appState.setWarning(processResp.warning || ""); appState.setLoadingStep("Recommending");
const recResp = await recommend({ user_id: userId, measurements: processResp.measurements,
measurement_id: processResp.measurement_id, brands
}); const normalized = recResp.recommendations || [recResp];
appState.setRecommendations(normalized); const history =
await getUserHistory(userId, 5); appState.setHistory(history);
return { processResp, recommendations: normalized, history };
},
[appState]
);
return { analyzeFit };
}
```

APPENDIX 2 : SCREENs

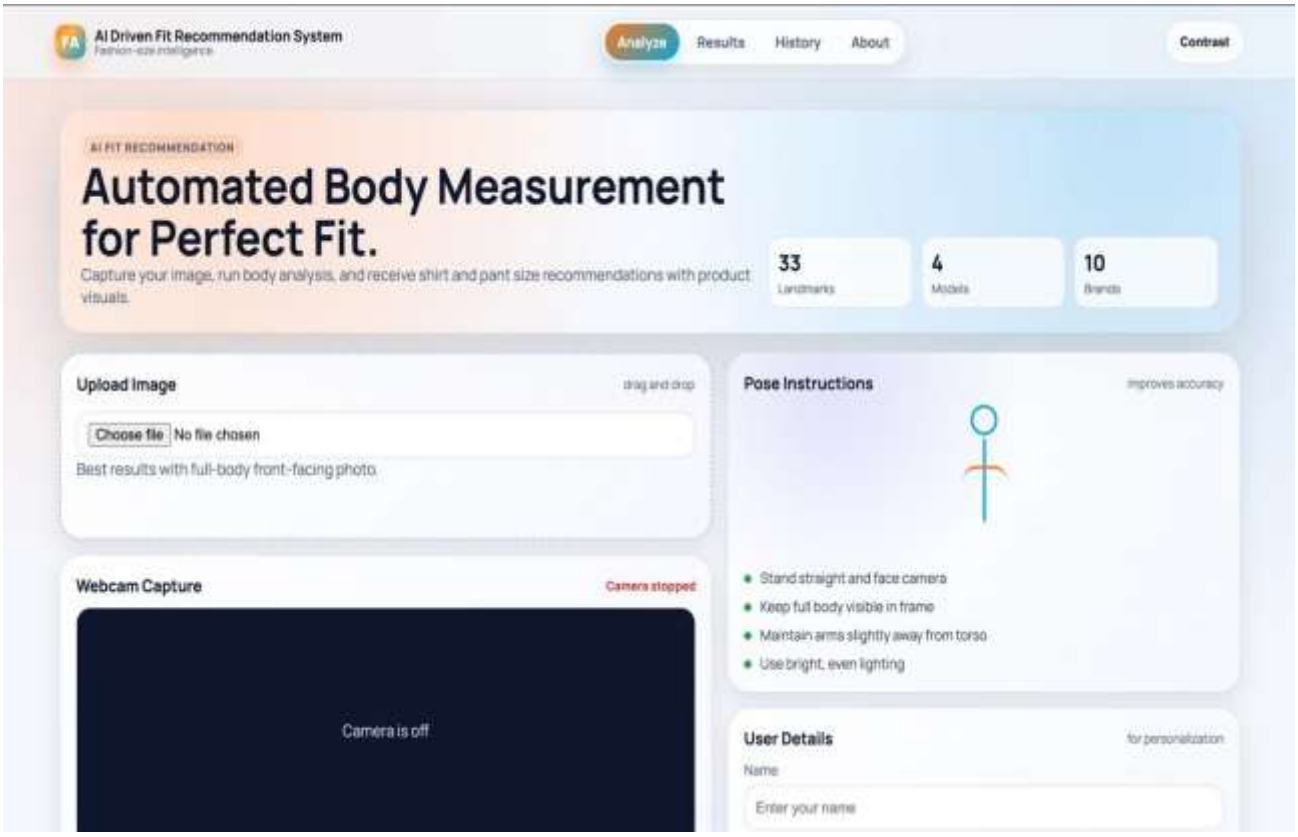


Fig 4.6.1 : User DashBoard

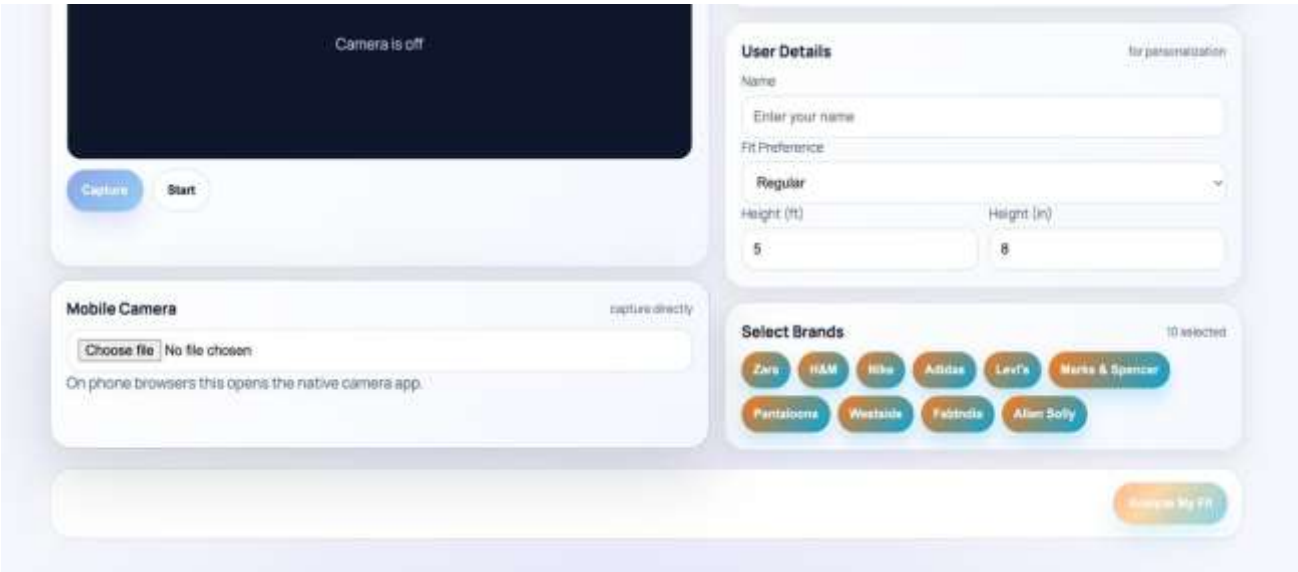


Fig 4.6.2 :Manual Input of height and other details

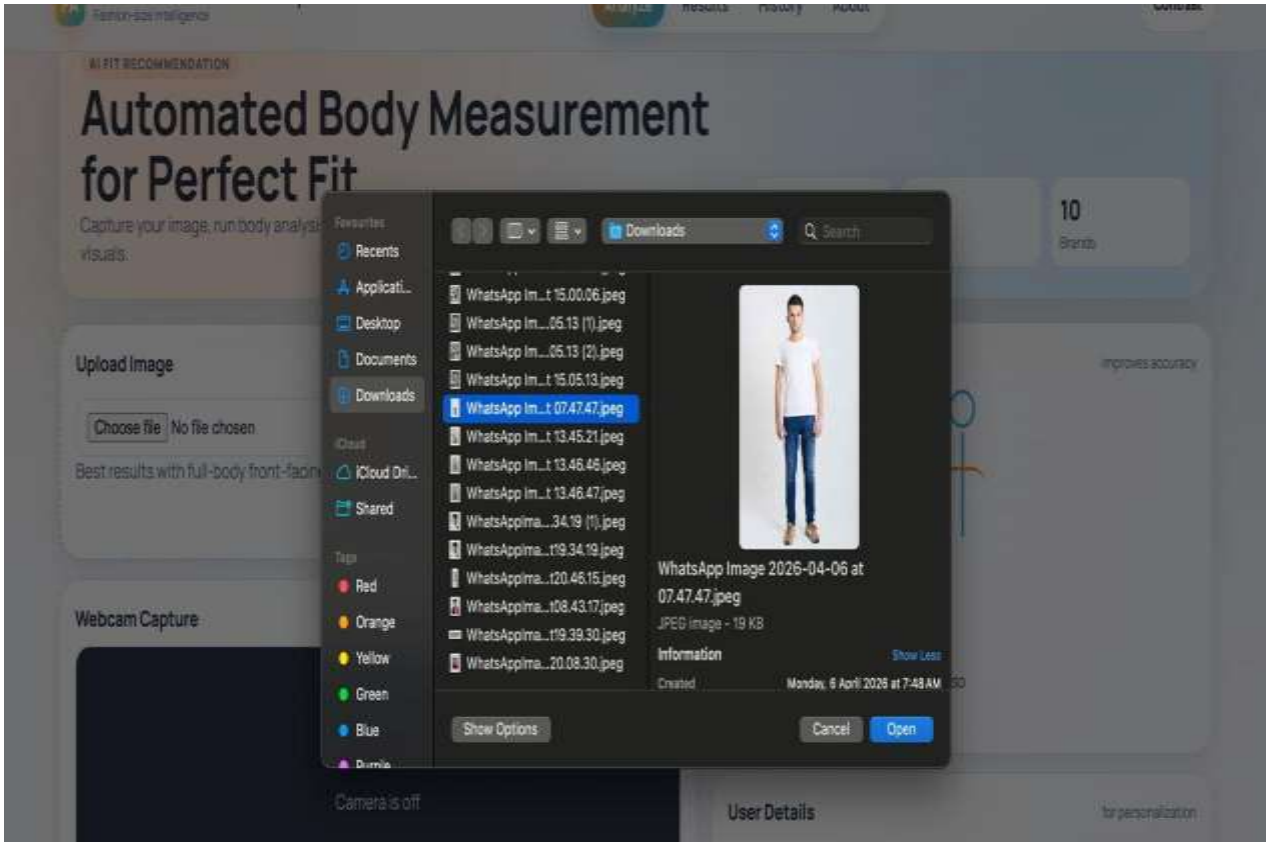


Fig 4.6.3 : Select the image

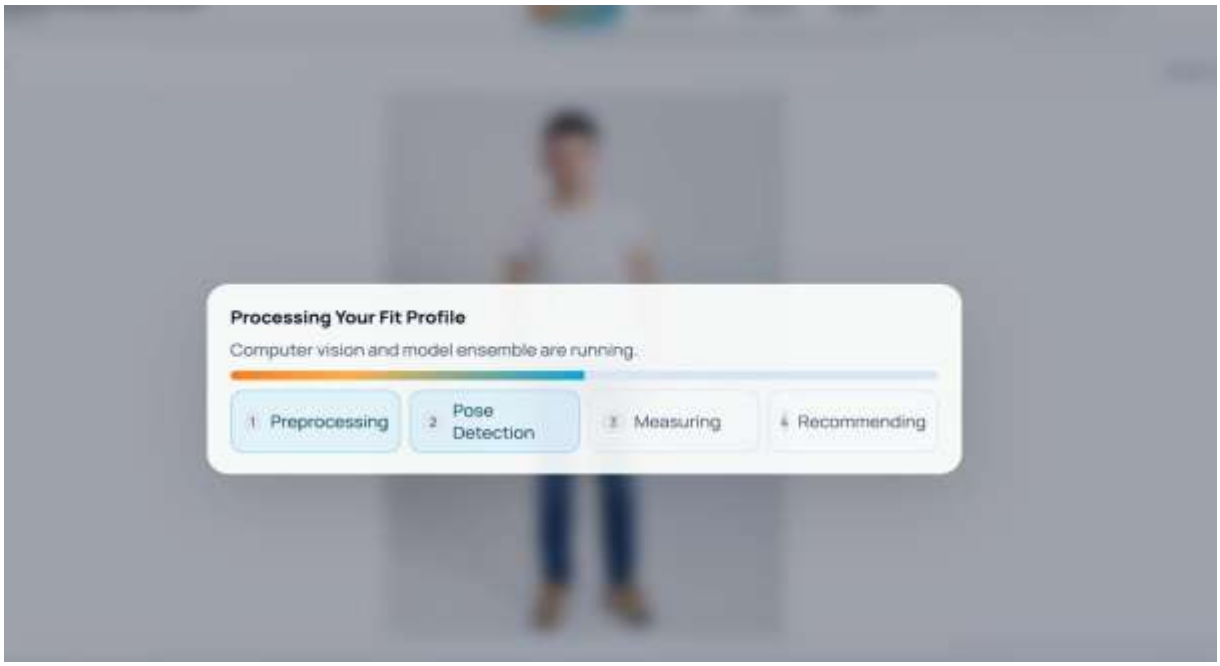


Fig 4.6.4 :Processing Steps of image

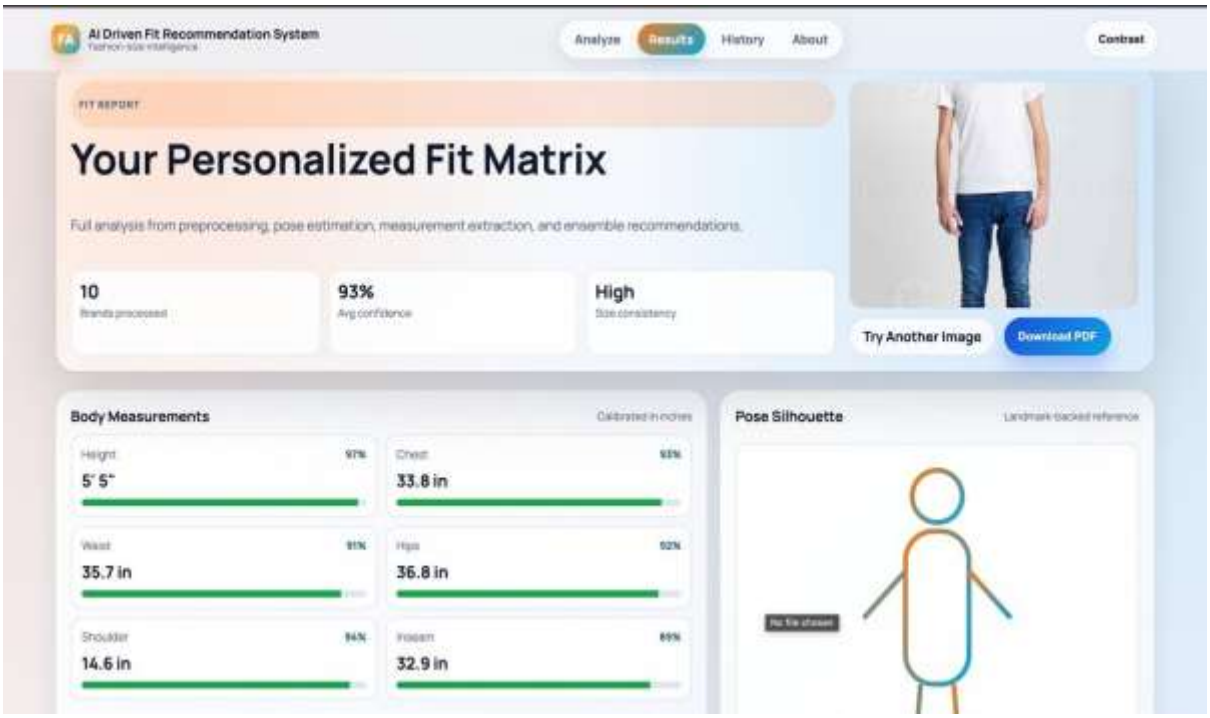


Fig 4.6.5 : Body Measurements

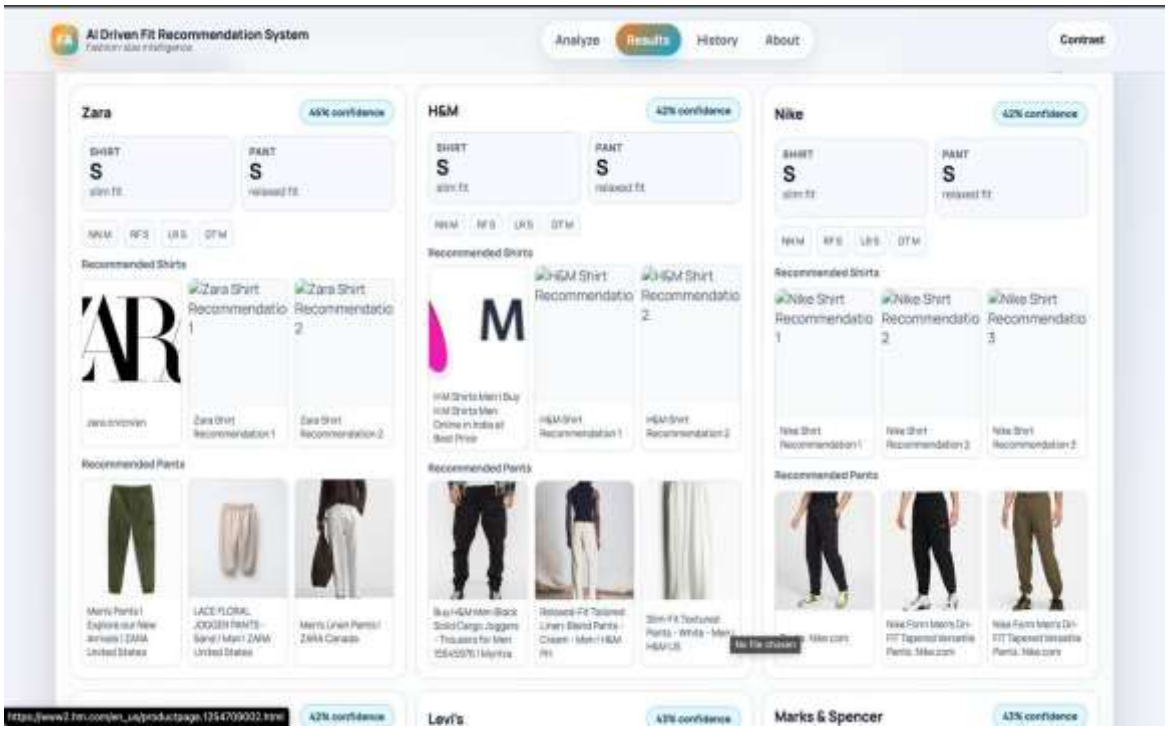
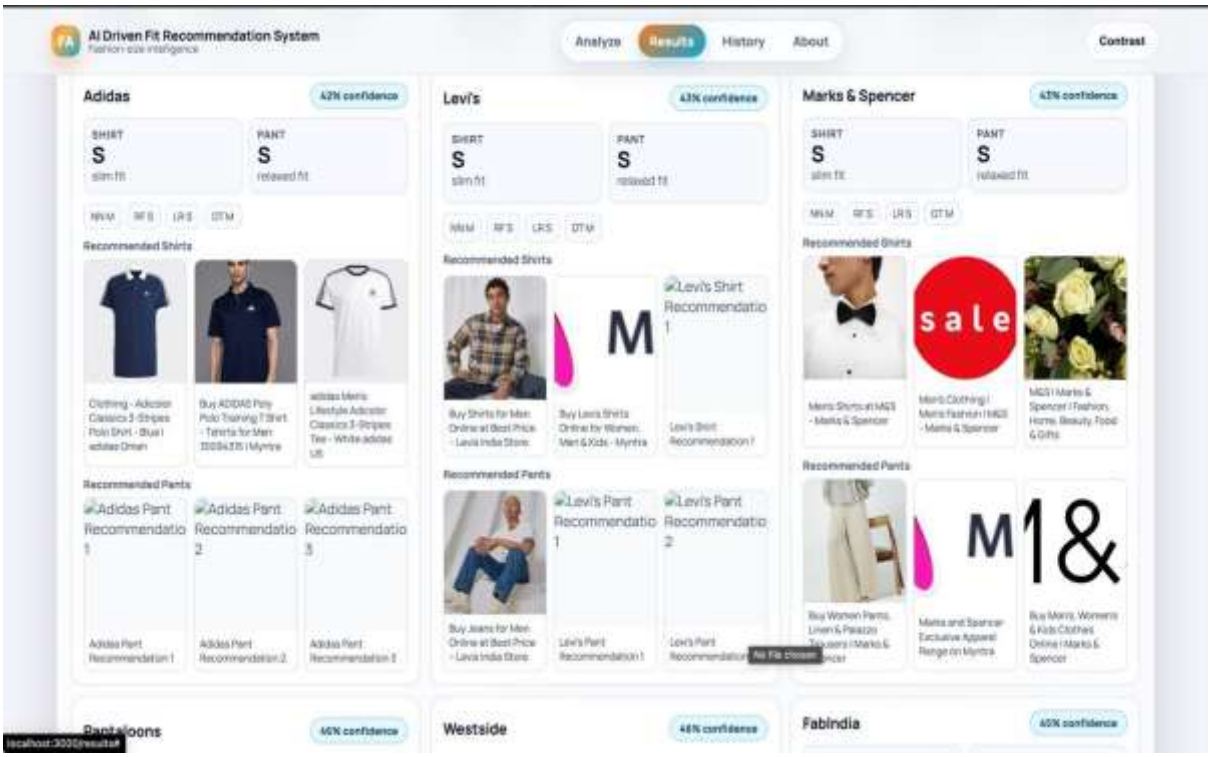
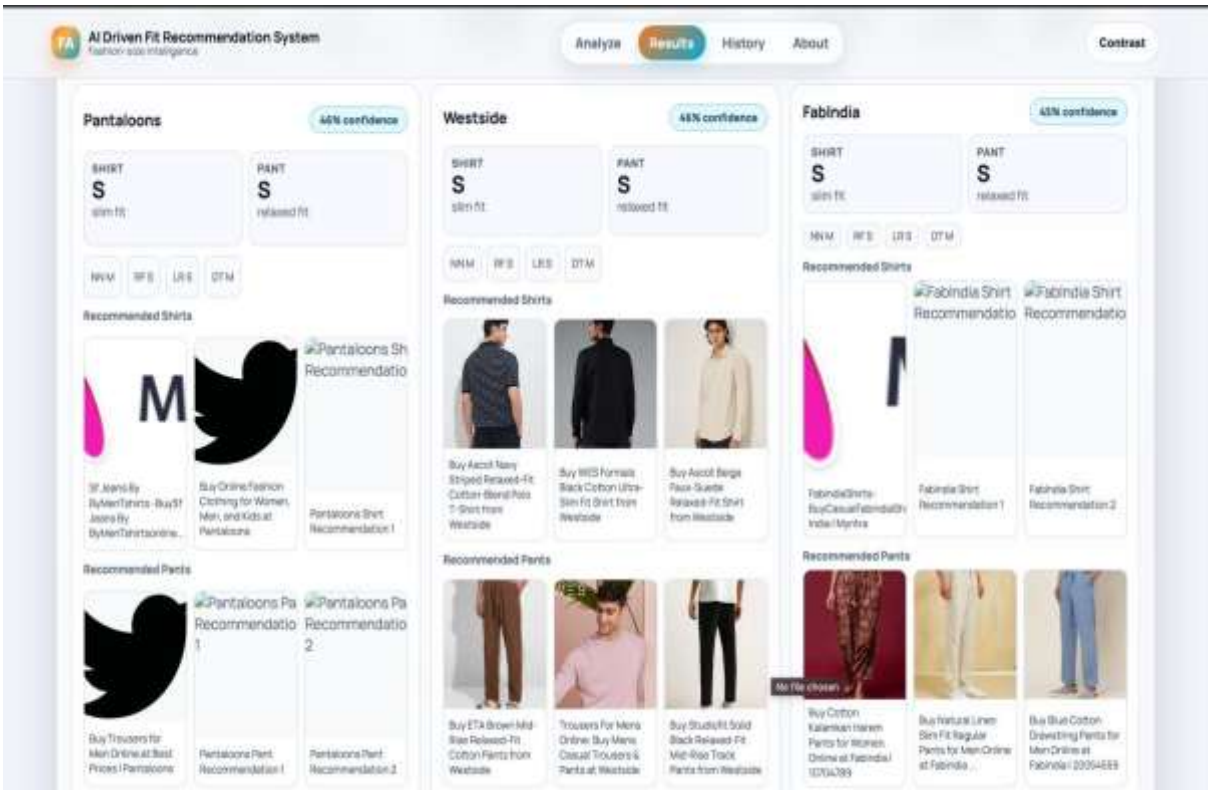


Fig 4.6.6 :Recommendation System


AI Driven Fit Recommendation System
Analyze
Results
History
About
Contrast
Adidas
42% confidence
SHIRT
S
slim fit
PANT
S
relaxed fit
Recommended Shirts
Recommended Pants
Levi's
43% confidence
Marks & Spencer
43% confidence
sale
Pantaloons
46% confidence
Westside
46% confidence
Fabindia
45% confidence


AI Driven Fit Recommendation System
Analyze
Results
History
About
Contrast
Pantaloons
46% confidence
SHIRT
S
slim fit
PANT
S
relaxed fit
Recommended Shirts
Recommended Pants
Westside
46% confidence
Fabindia
45% confidence

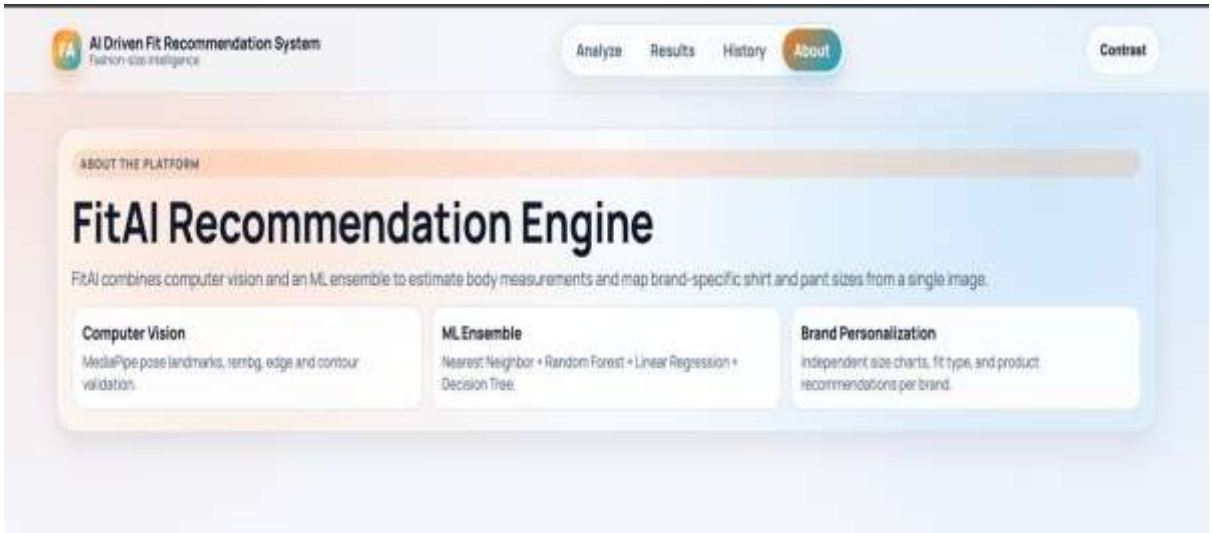


Fig 4.6.7 : Recommendation Engine

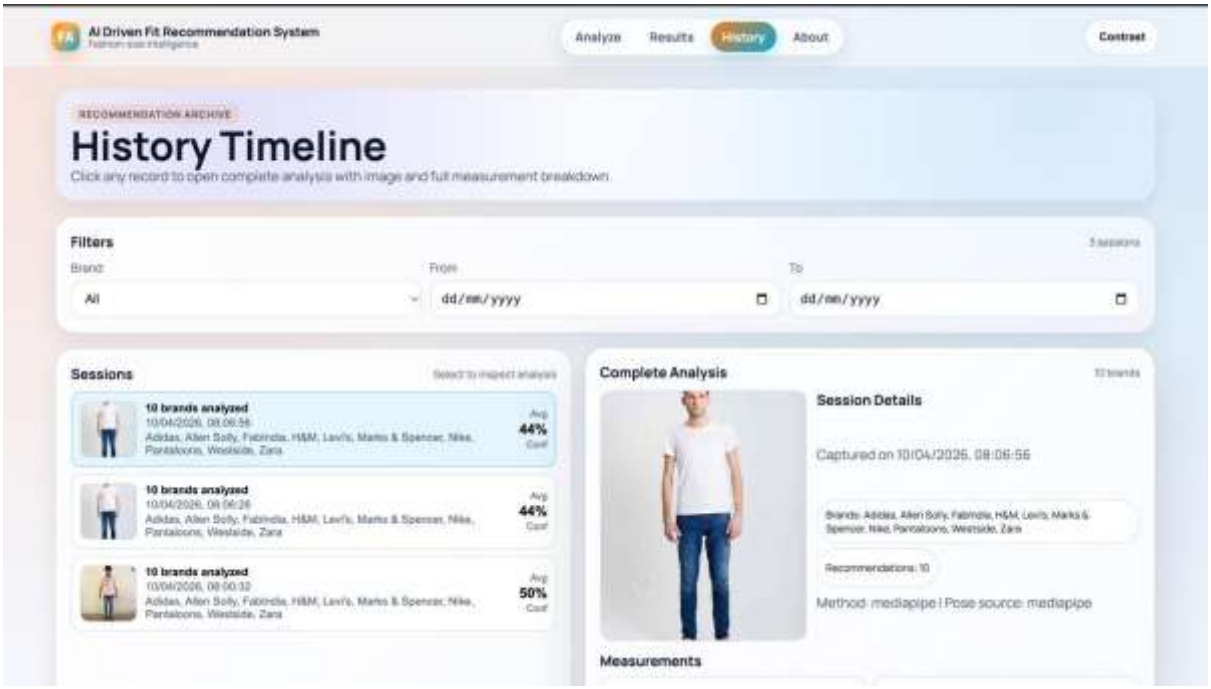


Fig 4.6.8: History Timeline